

# SWAMINARAYAN UNIVERSITY



**FACULTY OF ENGINEERING**

**DIPLOMA ENGINEERING**

**DEPARTMENT OF COMPUTER ENGINEERING**

**Basic Of Operating System  
(DE104304)**

**3<sup>rd</sup> SEMESTER**

# Unit-1 Introduction of Operating System

## ❖ **Fundamental Goals of Operating system:**

The operating system controls and coordinates the user of the hardware among the various application programs for the various users.

It hides complexity of its hardware from user and provide easy interface.

An operating system is a control program. It controls the execution of user programs to prevent errors and improper use of the computer.

The **Primary goal** of operating system is **convenience for the user**. The operating system makes the use of system easier.

The secondary goal of operating system is **efficient operation of the computer system**.

A Computer system can be divided into four components:

1. Hardware
2. Operating System
3. Application Programs
4. Users

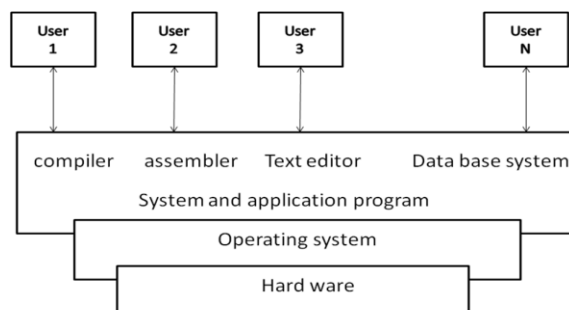


Fig :Component of Computer System

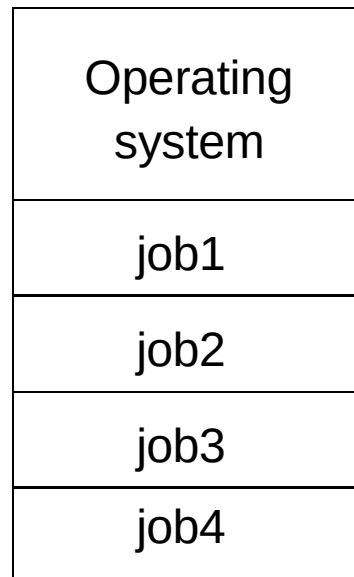
1. **Hardware** :The **hardware** of computer system includes the **Central Processing Unit (CPU), the memory, and the input/output (I/O) devices**. They are the basic resources.
2. **Application program** includes compilers, database system, games, and business programs. They define the way in which the hardware resources are to be used to solve the computing problems of the users.
3. **Users** :There are many different users trying to solve different problems using different application programs.
4. **Operating system** :The operating system controls and coordinates the user of the hardware among the various application programs for the various users.

---

## ❖ Overview of Operating Systems

### ❖ Multi programming Operating System

Multiprogramming **increases CPU utilization** by organizing jobs so that the CPU always has one job to execute.



The operating system keeps several jobs in memory simultaneously. This set of jobs is a subset of the jobs kept in the **job pool** the number of jobs that can be kept simultaneously in memory is usually much smaller than the number of jobs that can be in the job pool.

Operating system picks and begins to execute one of the jobs in memory and the other job may have to wait for some task, such as I/O operation.

In multiprogramming system, the CPU simply switches to, and executes another job. When that job needs to wait, the CPU is switched to another job.

Eventually the first job finishes waiting and gets CPU back .CPU has always at least one job to execute.

In multiprogramming operating system all the jobs that enter the system are kept in the pool. This pool consists of all processes residing on disk awaiting allocation of main memory.

When the operating system selects a job pool, it loads that job into main memory is called **job scheduling**.

If several jobs are ready to run at the same time, the system must choose among them, this decision is called **CPU scheduling**.

### Required feature to do multi programming

To store several programs in memory simultaneously, better memory management & protection are required.

---

As main memory is limited, virtual memory and swapping are used to accommodate more than one program in memory.

CPU scheduling should be sophisticated, to provide fairness to all programs.

Disk management and file system is complex now.

Synchronization and communication among running programs and problems like dead lock should be overcome.

## ❖ Time Sharing Operating System

It is a logical extension of multiprogramming operating system.

The ability to execute more than one task at the same time, a task being a program is called **multitasking**.

The terms multitasking and multiprocessing are often used interchangeably although multiprocessing implies that more than one CPU is involved.

In multitasking, only one CPU is involved, but it switches from one program to another so quickly that each user may interact with each program while it is running. So it gives the appearance of executing all of the programs at the same time.

A **time – shared operating system** uses CPU scheduling and multiprogramming to provides each user with a small portion of a time – shared computer. Each user has at least one separate program in memory.

A program in execution is called process.

**Time slice** is amount of that is given to process to execute on processor (CPU).

A time – Sharing operating system must have memory management, synchronization, communication, protection file system and disk management.

There are two basic types of multitasking:

- 1) Pre emptive multitasking (forcibly)
- 2) Non-preemptive cooperative. ( co operative or voluntarily)

In **preemptive** - a program is allowed to execute only for some maximum time duration, after this time duration CPU is forcibly taken away from the program.

- Minimize the response time for user.

**Ex:** OS/2, Windows 95, Windows NT, the Amiga operating system and UNIX use

In **Non-preemptive** multitasking, each program can control the CPU for as long as it needs it.

- If a program is not using the CPU .However, it can allow another program to use it temporarily.

**Ex:** Microsoft Windows 3.x and the MultiFinder (for Macintosh computers)

---

## ❖ Real Time Operating System

A real-time operating system is used where there are rigid time requirement on the operation of a processor or the flow of data, and thus is often used as a control device in a dedicated application.

RTOS has very little user-interface ,capability and no end user utilities. RTOS always controls inputs and respond within defined time constraint.

Systems that control scientific experiments, medical imaging systems, industrial control systems, and some displays, automobile-engine fuel-injection system are real-time system.

A real-time operating system has well-defined, fixed time constraints processing must be done with the defined constraints, or the system will fail.

- **There are two flavors of real-time systems:**

1. **Hard real-time:** A hard real-time system guarantees that critical tasks complete on time. This goal requires that all delays in the system be bounded , from the retrieval of stored data to the time that takes the operating system to finish any request made of it.

- Secondary storage of any sort is usually limited or missing , with data being stored in short-term memory or in read-only memory(ROM).

- Most advanced operating system features such as virtual memory is not found in real-time system.

Ex: oil refinery (plant),ATM

2. **Soft real-time system:** A less restrictive type of real-time operating system is a soft real-time system, where a critical real-time task gets priority over other tasks , and retains that priority until it completes.

- Soft real-time system have more limited utility that hard real-time system.

- They are risky if used for industrial control and robotics because of their lack of deadline support.

- They are useful in multimedia , virtual reality, and advanced scientific projects such as undersea exploration and planetary rovers.

Ex: digital audio ,multimedia

## ❖ Multithreading Operating System

- A thread also called a **lightweight process (L W P)** is a basic unit of CPU utilization; it comprises a thread ID, a program counter, a register set, and a stack.

- Thread is a flow of control within a process.

- It shares with other threads belonging to the same process its code section, data section and other operating system resources.

- Process is a **heavy** **weight process**, has a single thread of control.

- If the process has multiple threads of control, it can do more than one task at a time.

---

---

**For eg .**A web browser might have one thread display images or text while another thread retrieves data from the network.

A word process may have a thread for displaying graphics, another thread for reading keystroke from users, and a third thread for performing spelling and grammar checking.

### **Advantage of multithread programming**

#### **1. Responsiveness:**

**Multithreading** an interactive application may allow a program to continue running even if part of it is blocked or is performing lengthy operation, thereby increasing responsiveness to the user.

#### **2. Resource sharing**

Threads share the memory and resources of the process to which they belong. The Benefit of cost sharing is that it allows an application to have several different threads of activity all within the same address space.

#### **3. Economy**

Allocating memory and resources for process creation is costly. Threads share resources of the process to which they belong, it is more economical to create and context switch threads.

#### **4. Utilization of multiprocessor architecture**

Multithreading can be greatly increased in multiprocessor architectures, where each thread may be running in parallel on a multiprocessor.

A single-threaded process can only run on one CPU, no matter how many are available. Multithreading on a multi-CPU machine increases concurrency.

### **❖ Distributed Operating System**

A distributed system is a collection of loosely coupled processors interconnected by a communication network.

In distributed operating system **processors do not share memory or a clock**. Instead, each processor has its own local memory and the processors communicate with one another through various communication lines, such as high-speed buses or telephone-lines.

- These systems are usually referred to as **loosely coupled system(or distributed system)**.
- Distributed operating system evolved from computer networks in which a number of largely independent hosts are connected by communication links and protocols.
- A distributed operating system governs the operation of a distributed computer system and provides a virtual machine abstraction to its users. The key objective of distributed operating system is **transparency**.
- A distributed operating system has **less autonomous environment**. Use of such system types the commands and it runs but it is up to OS-level to decide which processor is allocated to him/her for performing task.

- 
- **User of such systems is not aware of the multiplicity of machine**, means he/she are not aware of the fact that multiple computers are being used. It seems like a virtual uniprocessor. Components and resource distribution should be hidden from users and application program unless they explicitly demand.
  - Distributed operating system usually **provides the means for system-wide sharing of resources, such as computational capacity, files, and I/O devices.**
  - Distributed operating system also may facilitate access to remote resources, Communication with remote processes and distribution of computations along with services necessary for pooling of shared system resources include global naming, distributed file system, and facilitate for distribution of computations, such as internodes process communication and remote procedure calls.

#### **Advantages of distributed systems:**

##### **1. Resource sharing:**

If a number of different sites are connected to one another, than a user at one site may be able to use the resources available at another.

##### **2. Reliability:**

If one site fails in a Distributed system, the remaining sites can continue operating, giving the system better reliability.

##### **3. Communication:**

Several sites are connected to one another by a communication network. The user at different sites has the opportunity to exchange information.

##### **4. Computation speeds up.**

In distributed system computation (work load) distributes on different machine (,thus it provides load balancing.)Overloaded systems work gets distributed on other system. so computation can be done speedily.

#### **❖ Operating System services**

The operating system provides an environment for the execution of programs. The operating system provides services to programs to the users of those programs .The operating system services are provided for the convenience of the programmer.

Operating system provides services to the user program and system.

#### **For user point of view services:**

**1. User Interface:** it provides interface to user so user can perform some action. It provides three types of interfaces,

**a) Command line Interface (CLI),** in which user can type command and methods for executing them.

---

b) **Batch interface**, in which commands and directives to control those commands are entered into files ,and those files are executed.

c) **Graphical User Interface (GUI)**,in which interface is a window system with pointing device to direct I/O ,chooses from menus, keyboard to enter text.

2. **Program execution:** The system must be able to load a program into memory and run it. The program must be able to end its execution either normally or abnormally (indicating error).

3. **I/O operations:** A running program may require I/O involves a file or an I/O device. For specific devices, special functions may be desired. For efficiency protection , users usually cannot control I/O devices directly. The operating system provides a means to do I/O.

4. **File-system manipulation:** The programs need to read and write files and programs must also create and delete files. The operating system maintains the file system.

5. **Communications:** In many conditions the process needs to exchange information with another process. Such communications can occur in two ways:

a) The fist takes place between processes that are executing on the same computer.

b) The second takes place between processes that are executing on different computer systems that are tied together by a network.

Communications may be implemented via **shared memory or via message passing** , in which the information is moved between processes by operating system.

6. **Error Detection:** The operating system constantly needs to be aware of possible errors. Errors may occur in the CPU and memory hardware, in I/O devices and in user programs.

For each type of errors, the operating system should take the proper action to ensure correct and consistent computing.

### **For System point of view services**

Operating a system function that exists not for helping the user, but rather ensuring the efficient operation of the system.

**1. Resource allocation:** Operating system manages many resources like CPU cycles, main memory, and file storage. Resources are allocated to multiple users.

Some resources have special allocation code, whereas others may have general request and release code.

**For eg.** In determining how best to use the CPU, the operating system have CPU- scheduling routines that takes account the speed of the CPU,the jobs that must be executed, the number of registers.

**2. Accounting:** The tracking of which user use how many wnd which kind of computer resources can be used for billing or simply accumulating usage statistics.

Usage statistics may be a valuable tool for researchers who wish to reconfigure the system to improve computing services.

---

**3. Protection:** Protection involves ensuring that all access to system resources is controlled.

## ❖ Generations of Operating System

### The First Generation (1940 to early 1950s)

When the first electronic computer was developed in 1940, it was created without any operating system. In early times, users have full access to the computer machine and write a program for each task in absolute machine language. The programmer can perform and solve only simple mathematical calculations during the computer generation, and this calculation does not require an operating system.

### The Second Generation (1955 - 1965)

The first operating system (OS) was created in the early 1950s and was known as **GMOS. General Motors** has developed OS for the **IBM** computer. The second-generation operating system was based on a single stream batch processing system because it collects all similar jobs in groups or batches and then submits the jobs to the operating system using a punch card to complete all jobs in a machine.

### The Third Generation (1965 - 1980)

During the late 1960s, operating system designers were very capable of developing a new operating system that could simultaneously perform multiple tasks in a single computer program called multiprogramming. The introduction of **multiprogramming** plays a very important role in developing operating systems that allow a CPU to be busy every time by performing different tasks on a computer at the same time.

### The Fourth Generation (1980 - Present Day)

The fourth generation of operating systems is related to the development of the personal computer. However, the personal computer is very similar to the minicomputers that were developed in the third generation. The cost of a personal computer was very high at that time; there were small fractions of minicomputers costs. Currently, most Windows users use the Windows 10 operating system. Besides the Windows operating system, Apple is another popular operating system built in the 1980s, and this operating system was developed by Steve Jobs, a co-founder of Apple. They named the operating system Macintosh OS or Mac OS.

---

## ❖ Case Study

### 1. Linux

Linux is a family of free and open-source operating systems based on the Linux kernel. Operating systems based on Linux are known as *Linux distributions* or *distros*. Examples include Debian, Ubuntu, Fedora, CentOS, Gentoo, Arch Linux, and many others.

The Linux kernel has been under active development since 1991, and has proven to be extremely versatile and adaptable. You can find computers that run Linux in a wide variety of contexts all over the world, from web servers to cell phones. Today, 90% of all cloud infrastructure and 74% of the world's smartphones are powered by Linux.

However, newcomers to Linux may find it somewhat difficult to approach, as Linux filesystems have a different structure than those found on Windows or MacOS. Additionally, Linux-based operating systems depend heavily on working with the command line interface, while most personal computers rely on graphical interfaces.

This guide serves as an introduction to important

The terms “terminal,” “shell,” and “command line interface” are often used interchangeably, but there are subtle differences between them:

- *A terminal* is an input and output environment that presents a text-only window running a shell.

*A shell* is a program that exposes the computer's operating system to a user or program. In Linux systems, the shell presented in a terminal is a command line interpreter.

*A command line interface* is a user interface (managed by a command line interpreter program) which processes commands to a computer program and outputs the results.

### 2. Latest Windows Operating System

**Windows 11** is the latest major release of Microsoft's Windows NT operating system, released in October 2021. It is a free upgrade to its predecessor, Windows 10 (2015), and is available for any Windows 10 devices that meet the new Windows 11 system requirements.

## **Unit-2 Process Management**

### **❖ Overview of the Process & threads:**

#### **Process:-**

In computing, a process is an instance of a computer program that is being executed.

It contains the program code and its current activity. Whereas a program is a set of Instructions

Process is a part of code which under execution or in detail u can say that in main memory whereas a program is a complete task or work to be done

A process is a program in execution. it also includes the current activity, as represented by the value of program counter and the contents of the program counter.

A process includes a process stack which contains the temporary data and a data section which contains the global variables. while a program is a sequence of instructions written in a manner to obtain the desired output.

#### **Thread:-**

Thread is part of program that can be executed simultaneously independently from the other part of a program. The Operating System has a scheduler for each thread (process) that is currently running. It divides up time slices for each of them, which are executed in the order that Operating System seems fit. It simply runs each one in some arbitrary order for a set number of milliseconds and then switches between them constantly.

#### **User Threads**

They are supported above the kernel and are implemented by a thread library at the user level.

The library provides support for thread creation, scheduling and management with no support from kernel. Kernel is unaware of user-level threads; all thread creation and scheduling is done in the user space so they are fast to create and manage.

User-thread libraries include POSIX P-threads, Mach C-Thread and solaris UI-threads.

#### **Kernel Threads**

Kernel threads are supported by Operating System: the kernel performs thread creation, scheduling and management in kernel space. Kernel threads are slower to create and manage.

Windows NT, Windows 2000, Solaris 2, BeOS, and Tru64 UNIX support kernel threads.

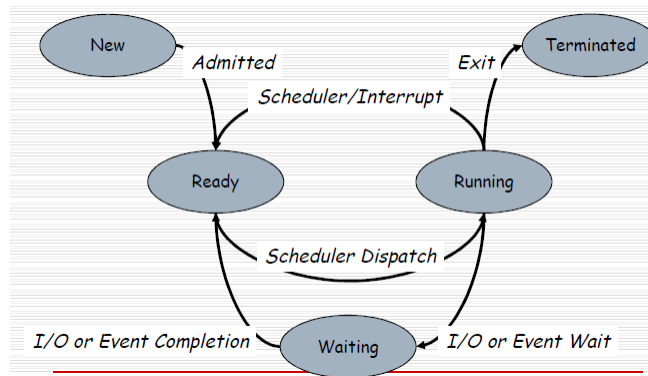
### **❖ Process Life Cycle/ Process States**

In computing, a process is an instance of a computer program that is being executed.

It contains the program code and its current activity.

A process includes a process stack which contains the temporary data and a data section which contains the global variables.

A process moves through various states during its life:



### 1. New

A program which is going to be picked up by the OS into the main memory is called a new process.

### 2. Ready

Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory.

The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.

### 3. Running

One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

### 4. Block or wait

From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.

When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

### 5. Completion or termination

When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

---

## ❖ Process Control Block

Each process is represented by a *process control block* (PCB) which contains information such as: state of the process (ready, running, blocked) ,register values ,priorities and scheduling information memory management information, accounting information such as open files allocated resources

The Attributes of the process are used by the Operating System to create the process control block (PCB) for each of them. This is also called context of the process. Attributes which are stored in the PCB are described below.

### 1. Process ID

When a process is created, a unique id is assigned to the process which is used for unique identification of the process in the system.

### 2. Program counter

A program counter stores the address of the last instruction of the process on which the process was suspended. The CPU uses this address when the execution of this process is resumed.

### 3. Process State

The Process, from its creation to the completion, goes through various states which are new, ready, running and waiting. We will discuss about them later in detail.

### 4. Priority

Every process has its own priority. The process with the highest priority among the processes gets the CPU first. This is also stored on the process control block.

### 5. General Purpose Registers

Every process has its own set of registers which are used to hold the data which is generated during the execution of the process.

### 6. List of open files

During the Execution, Every process uses some files which need to be present in the main memory. OS also maintains a list of open files in the PCB.

### 7. List of open devices

OS also maintain the list of all open devices which are used during the execution of the process.

---

## ❖ Scheduling Criteria:

**CPU utilization** -keep the CPU as busy as possible.

**Throughput** -Number of processes that complete their execution per time unit.

**Arrival Time**-The time at which the process enters into the ready queue is called the arrival time.

**Burst Time**- The total amount of time required by the CPU to execute the whole process is called the Burst Time. This does not include the waiting time. It is confusing to calculate the execution time for a process even before executing it hence the scheduling problems based on the burst time cannot be implemented in reality.

**Completion Time**-The Time at which the process enters into the completion state or the time at which the process completes its execution, is called completion time.

**Turnaround time**-The total amount of time spent by the process from its arrival to its completion, is called Turnaround time.

**Waiting Time**-The Total amount of time for which the process waits for the CPU to be assigned is called waiting time.

**Response Time**-The difference between the arrival time and the time at which the process first gets the CPU is called Response Time.

## ❖ Scheduling Algorithms

### 1. First Come First Serve

**First come first serve (FCFS)** scheduling algorithm simply schedules the jobs according to their arrival time. The job which comes first in the ready queue will get the CPU first. The lesser the arrival time of the job, the sooner will the job get the CPU. FCFS scheduling may cause the problem of starvation if the burst time of the first process is the longest among all the jobs.

#### *Advantages of FCFS*

- Simple
- Easy
- First come, First serv

#### *Disadvantages of FCFS*

1. The scheduling method is non preemptive, the process will run to the completion.

2. Due to the non-preemptive nature of the algorithm, the problem of starvation may occur.
3. Although it is easy to implement, but it is poor in performance since the average waiting time is higher as compare to other scheduling algorithms.

### Example

Let's take an example of The FCFS scheduling algorithm. In the Following schedule, there are 5 processes with process ID **P0, P1, P2, P3 and P4**. P0 arrives at time 0, P1 at time 1, P2 at time 2, P3 arrives at time 3 and Process P4 arrives at time 4 in the ready queue. The processes and their respective Arrival and Burst time are given in the following table.

The Turnaround time and the waiting time are calculated by using the following formula.

Turn Around **Time** = **Completion** Time - Arrival Time

Waiting **Time** = **Turnaround** time - Burst Time

The average waiting Time is determined by summing the respective waiting time of all the processes and divided the sum by the total number of processes.

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| P0 | P1 | P2 | P3 | P4 |    |
| 0  | 2  | 8  | 12 | 21 | 33 |

| Process ID | Arrival Time(T0) | Burst Time(T1) | Completion Time(CT) | Turn Around Time(TAT=CT-T0) | Waiting Time(WT=TAT-T1) |
|------------|------------------|----------------|---------------------|-----------------------------|-------------------------|
| P0         | 0                | 2              | 2                   | 2                           | 0                       |
| P1         | 1                | 6              | 8                   | 7                           | 1                       |
| P2         | 2                | 4              | 12                  | 10                          | 6                       |
| P3         | 3                | 9              | 21                  | 18                          | 9                       |
| P4         | 6                | 12             | 33                  | 29                          | 17                      |

Average Waiting Time= $31/5 = 6.2$  ms

Average Turn around Time= $66/5 = 13.2$  ms

## 2. Shortest Job First

Till now, we were scheduling the processes according to their arrival time (in FCFS scheduling). However, SJF scheduling algorithm, schedules the processes according to their burst time.

In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.

However, it is very difficult to predict the burst time needed for a process hence this algorithm is very difficult to implement in the system.

### *Advantages of SJF*

1. Maximum throughput
2. Minimum average waiting and turnaround time

### *Disadvantages of SJF*

1. May suffer with the problem of starvation
2. It is not implementable because the exact Burst time for a process can't be known in advance.

There are different techniques available by which, the CPU burst time of the process can be determined. We will discuss them later in detail.

|   |    |    |    |    |    |    |
|---|----|----|----|----|----|----|
|   | P1 | P3 | P2 | P5 | P4 |    |
| 0 | 1  | 8  | 10 | 13 | 21 | 31 |

### *Example*

In the following example, there are five jobs named as P1, P2, P3, P4 and P5. Their arrival time and burst time are given in the table below.

| PID | Arrival Time | Burst Time | Completion Time | Turn Around Time | Waiting Time |
|-----|--------------|------------|-----------------|------------------|--------------|
| 1   | 1            | 7          | 8               | 7                | 0            |
| 2   | 3            | 3          | 13              | 10               | 7            |
| 3   | 6            | 2          | 10              | 4                | 2            |
| 4   | 7            | 10         | 31              | 24               | 14           |
| 5   | 9            | 8          | 21              | 12               | 4            |

---

Since, No Process arrives at time 0 hence; there will be an empty slot in the **Gantt chart** from time 0 to 1 (the time at which the first process arrives).

According to the algorithm, the OS schedules the process which is having the lowest burst time among the available processes in the ready queue.

Average Waiting Time =  $27/5 = 5.4$  ms

Average Turn around Time =  $57/5 = 11.4$  ms

### 3. Round Robin

Round Robin scheduling algorithm is one of the most popular scheduling algorithm which can actually be implemented in most of the operating systems. This is the **preemptive version** of first come first serve scheduling.

#### *Advantages*

1. It can be actually implementable in the system because it is not depending on the burst time.
2. It doesn't suffer from the problem of starvation or convoy effect.
3. All the jobs get a fair allocation of CPU.

#### *Disadvantages*

1. The higher the time quantum, the higher the response time in the system.
2. The lower the time quantum, the higher the context switching overhead in the system.
3. Deciding a perfect time quantum is really a very difficult task in the system.

Here time quantum is 4ms

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| P1 | P2 | P3 | P4 | P5 | P1 | P6 | P2 | P5 |    |
| 0  | 4  | 8  | 11 | 12 | 16 | 17 | 21 | 23 | 24 |

The completion time, Turnaround time and waiting time will be calculated as shown in the table below.

As, we know,

Turn Around Time = Completion Time - Arrival Time

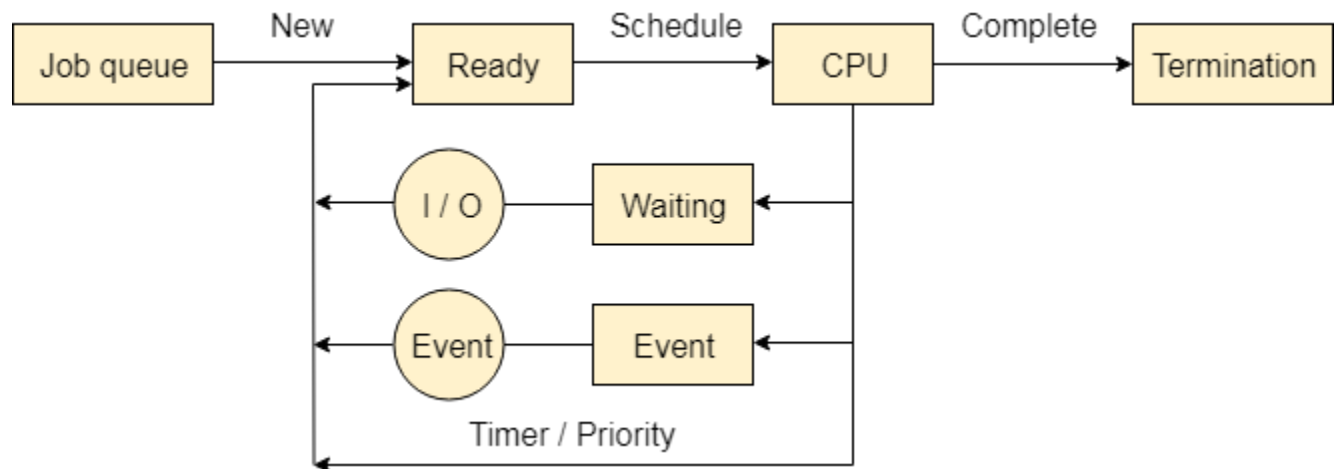
Waiting Time = Turn Around Time - Burst Time

| Process ID | Arrival Time | Burst Time | Completion Time | Turn Around Time | Waiting Time |
|------------|--------------|------------|-----------------|------------------|--------------|
| 1          | 0            | 5          | 17              | 17               | 12           |
| 2          | 1            | 6          | 23              | 22               | 16           |
| 3          | 2            | 3          | 11              | 9                | 6            |
| 4          | 3            | 1          | 12              | 9                | 8            |
| 5          | 4            | 5          | 24              | 20               | 15           |
| 6          | 6            | 4          | 21              | 15               | 11           |

$$\text{Average Waiting Time} = (12+16+6+8+15+11)/6 = 76/6 = 12.66$$

### ❖ Scheduling Queues

The Operating system manages various types of queues for each of the process states. The PCB related to the process is also stored in the queue of the same state. If the Process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.



There are the following queues maintained by the Operating system.

#### 1. Job Queue

In starting, all the processes get stored in the job queue. It is maintained in the secondary memory. The long term scheduler (Job scheduler) picks some of the jobs and put them in the primary memory.

## 2. Ready Queue

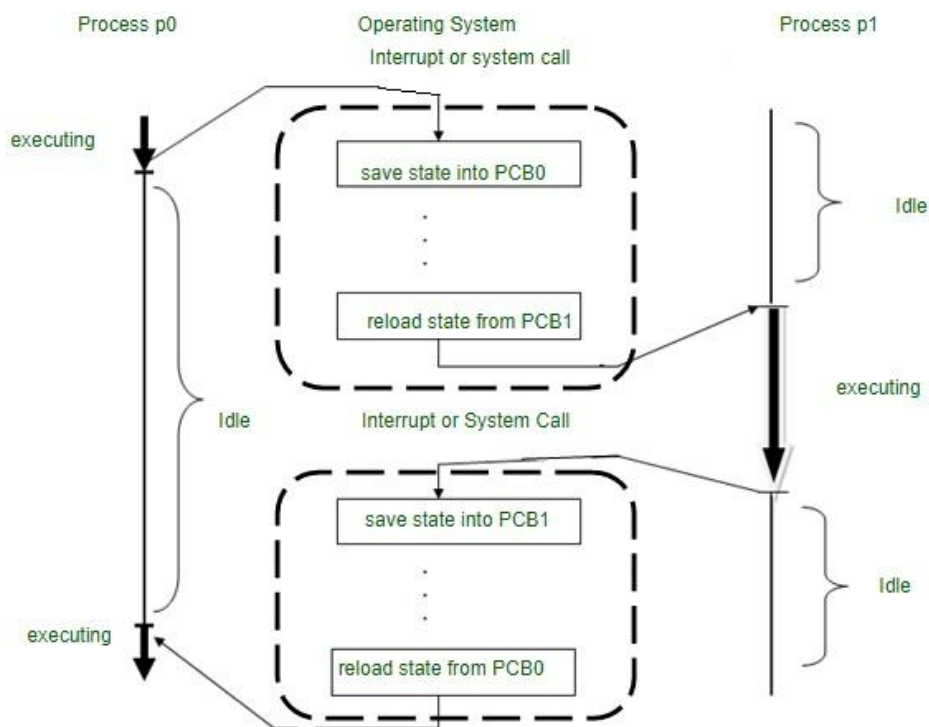
Ready queue is maintained in primary memory. The short term scheduler picks the job from the ready queue and dispatch to the CPU for the execution

## 3. Waiting Queue

When the process needs some IO operation in order to complete its execution, OS changes the state of the process from running to waiting. The context (PCB) associated with the process gets stored on the waiting queue which will be used by the Processor when the process finishes the IO.

## ❖ Context Switch

An operating system uses this technique to switch a process between states to execute its functions through CPUs. It is a process of saving the context (state) of the old process(suspend) and loading it into the new process(resume). It occurs whenever the CPU switches between one process and another. Basically, the state of CPU's registers and program counter at any time represent a context. Here, the saved state of the currently executing process means to copy all live registers to PCB (Process Control Block). Moreover, after that, restore the state of the process to run or execute next, which means copying live registers' values from PCB to registers.



❖

## ❖ Process Synchronization

Process synchronization is a mechanism to ensure systematic sharing of resources among concurrent (parallel) process.

Two issues related to Process synchronization:

- Two or more process should not come into each other's way
- Proper sequencing should be maintained during execution.

In IPC one problem arises called Race condition.

Race condition / Racing problem:

It is situation where two or more process are reading / writing some shared data and final result depends on relative order of their execution, is called race condition.

A=1000, At the end of two process A should be 1100. but if P0 and P1 permitted to execute in any arbitrary fashion then output will be not same.

| P0        | P1        |
|-----------|-----------|
| Read (A); | Read(A)   |
| A: A-100  | A :A+200; |
| Write (A) | Write (A) |

**Possibility : 1**

Here, Answer  
will be 1200

| P0        | P1        |
|-----------|-----------|
| Read (A); |           |
|           | Read(A)   |
| A: A-100  |           |
| Write (A) |           |
|           | A :A+200; |
|           | Write (A) |

**Possibility : 2**

Here, Answer  
will be 900

| P0        | P1        |
|-----------|-----------|
| Read (A); |           |
|           | Read(A)   |
|           | A :A+200; |
|           | Write (A) |
| A : A-100 |           |
| Write (A) |           |

Here two processes are reading and writing common variable 'A'. The final value depends on relative execution order of P0 and P1, such situation is called race condition.

This implies that concurrent process are racing with each other to access a shared resource in arbitrary order and procedure wrong final results, so race condition must be avoided.

---

## ❖ Critical Section

A critical section is a code segment of a process where the shared resources are accessed.

It is also known as critical region.

It is a part of a program or code segment of a process where a process may change common variables, updating a table, writing a file or accessing some physical device.

Only one process should be allowed to execute in its critical section.

OR

Only one process should be allowed to access a shared resource at a time.

It is the responsibility of an OS to ensure that no more than one process is present in its critical section simultaneously.

### **Requirement of critical section solution:**

An ideal critical section solution meet the following three condition:

#### **Mutual exclusion:**

At any time, at most one of the co-operating process should be executing in its critical section

#### **Progress:**

When no any process is running in its critical section and other process wants to enter its critical section, it should be allowed immediately.

If more than one process is waiting to enter their critical sections, one of them should be selected to enter.

This termed as requirement of progress, which must be met under all possible conditions.

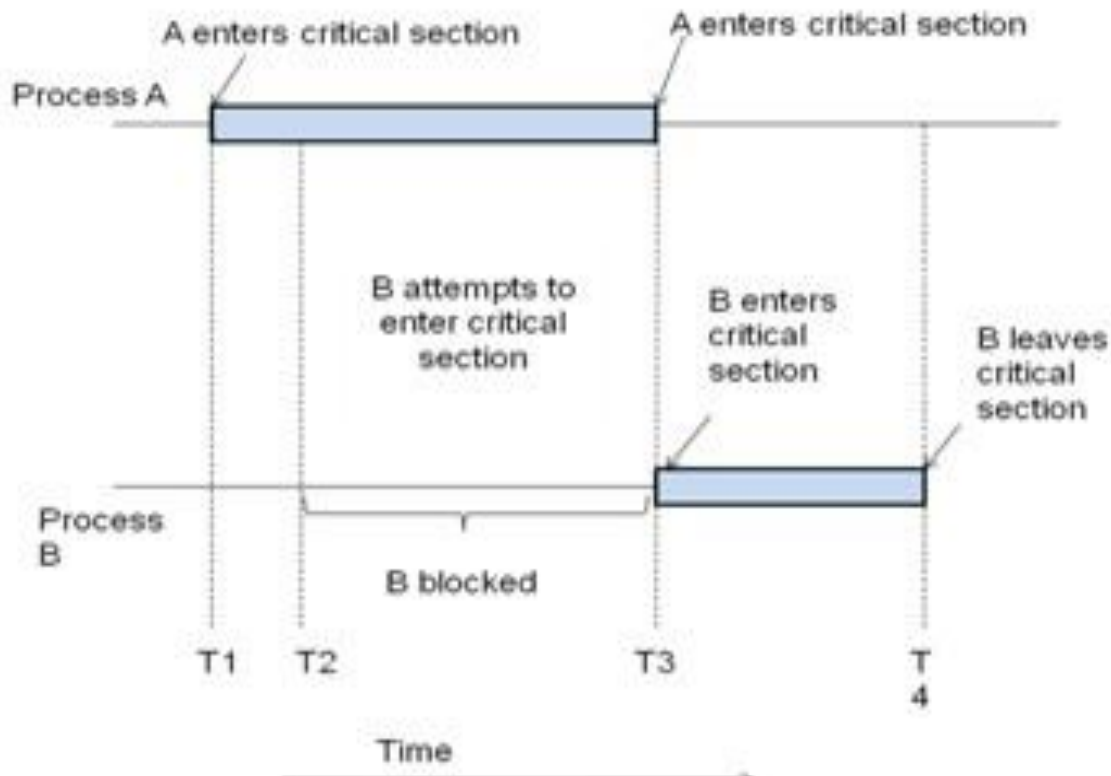
#### **Bounded waiting:**

When more than one process is waiting to enter their critical sections, one of them is selected to enter in critical section.

Other processes simply wait. But such type of waiting must be limited. There should not be starvation.

It should not be such that one process waits forever and other processes get entry in critical section more than once.

There should be some upper bound on the no, of times that other processes allowed to enter in their critical sections before a requesting process is granted to enter in its critical section.



## ❖ Mutual Exclusion

Mutual exclusion means only one process at a time can use a resource.

If some other process requests that resource, the requesting process must be waiting until the resource has been released.

We can deny this condition by simple protocol i.e., “convert the all non-sharable resources to sharable resources”. So this condition is not satisfied by the deadlock, then we can prevent the deadlock.

During concurrent execution of processes, processes need to enter the critical section (or the section of the program shared across processes) at times for execution. It might so happen that because of the execution of multiple processes at once, the values stored in the critical section become inconsistent. In other words, the values depend on the sequence of execution of instructions – also known as a race condition. The primary task of process synchronization is to get rid of race conditions while executing the critical section.

This is primarily achieved through mutual exclusion.

Mutual exclusion is a property of process synchronization which states that “no two processes can exist in the critical section at any given point of time”. The term was first coined by Dijkstra. Any process synchronization technique being used must satisfy the property of mutual exclusion, without which it would not be possible to get rid of a race condition.

---

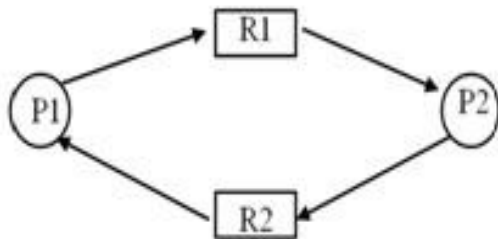
❖ **Deadlock** A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

A process request the resources, the resources are not available at that time, so the process enter into the waiting state.

The requesting resources are held by another waiting process, both are in waiting state, this situation is said to be “Deadlock”.

For example :

P1 and P2 are two processes, R1 and R2 are two resources. P1 request the resource R1, R1 held by process P2, P2 request the resource R2, R2 is held by P1, then both are entered into the waiting state. There is no work progress for process P1 and P2, it is also a Deadlock. Deadlock can be represented by Resource Allocation Graph.



### ❖ **Conditions for Deadlock**

Deadlock situation can arise if the following four conditions hold simultaneously (all at one time) in system.

A Deadlocked system must satisfy the following 4 conditions. These are:

**MUTUAL EXCLUSION:** Each resource can be assigned to exacty one process if any process requests resources which are not free then that process will wait until resource becomes free.

**HOLD AND WAIT:** A process must be holding at least one resource and waiting to acquire (get) additional resources that are currently being held by other process.

---

**NO PREEMPTION:** A process must be holding at least one resource and waiting to acquire (get) additional resources that are currently being held by other process. No preemption means resources are not released in the middle of the work, they released only after the process has completed its task.

**CIRCULAR WAIT:** There must be circular chain or two or more process . A set  $\{ P_1, P_2, \dots, P_n \}$  of waiting process must exist such that  $P_1$  is waiting for resource  $R_1$ ,  $P_2$  waits for resource  $R_2$ ,  $P_{n-1}$  waits for  $P_n$  and  $P_n$  waits for  $P_0$ . it said to be circular wait.

Process  $\{ P_0, P_1, P_2 \}$  and Resource  $\{ R_1, R_2, R_3, R_4 \}$

### ❖ Resource allocation graph

Deadlocks can be described accurately using a directed graph called a system resource allocation graph. This graph consists of a set of vertices  $V$  and a set of edges  $E$ .

The set of vertices  $V$  is partitioned into two different types of nodes :

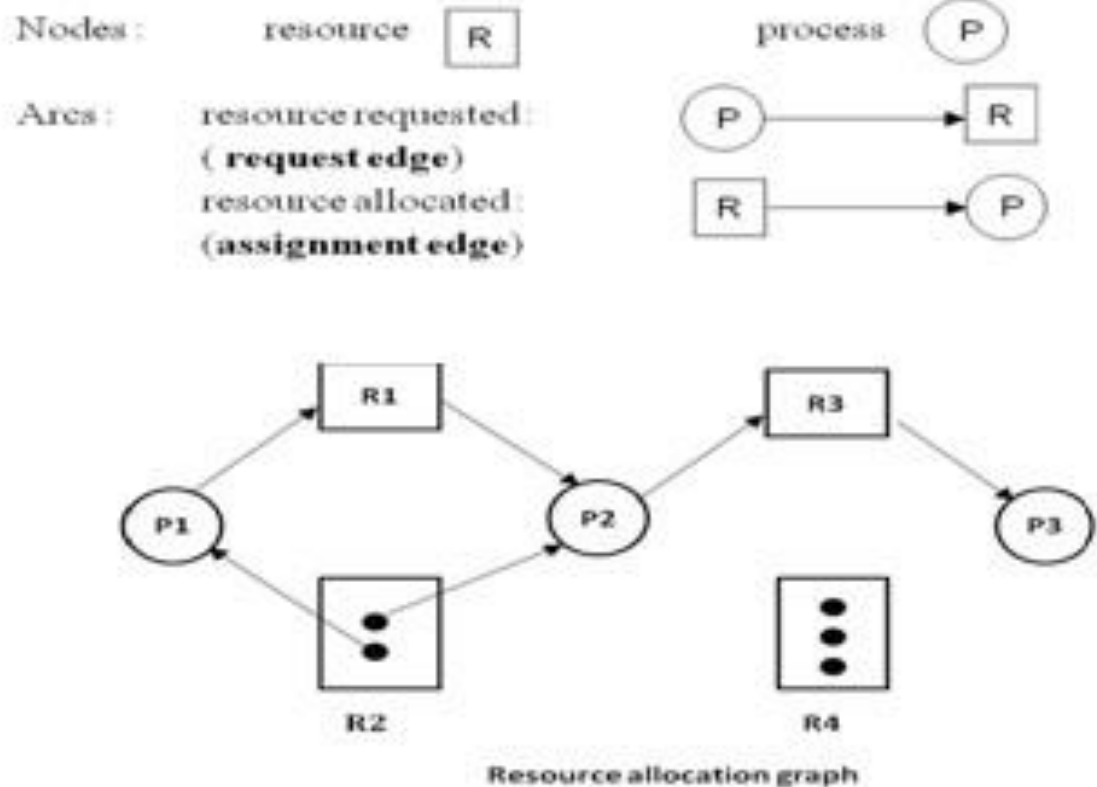
$P = \{ P_1, P_2, P_3, \dots, P_n \}$ , the set of all active processes in the system ;

- Process  $P_i$  represented using circle.

$R = \{ R_1, R_2, \dots, R_m \}$ , the set of all resource types in the system.

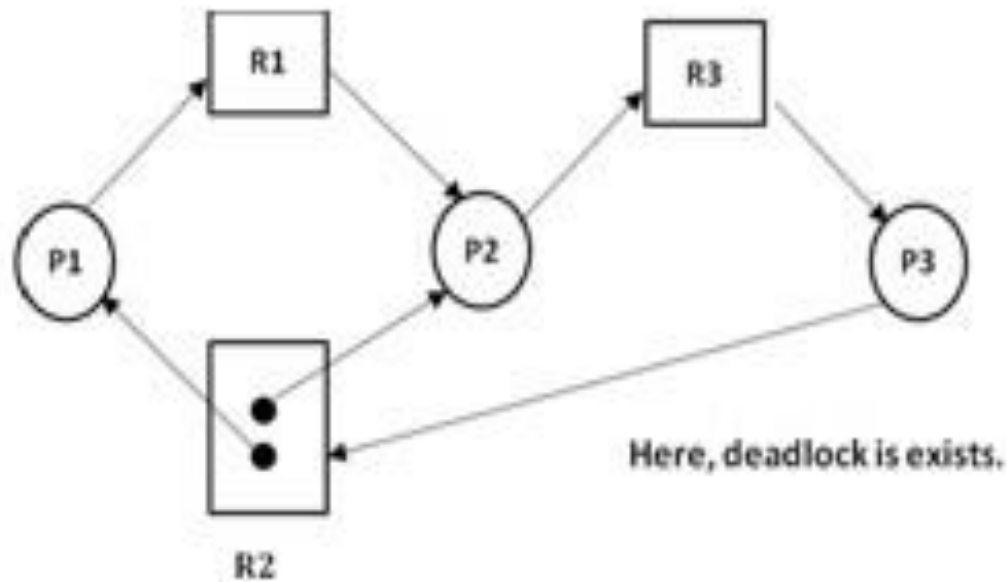
- Resource  $R_j$  represented using rectangle.
- If resource  $R_j$  may have more than instance as dot within circle.
- The set of edges  $E$  is in two different types :
- A directed edge from process  $P_i$  to resource type  $R_j$  is denoted by  $P_i \rightarrow R_j$ , it is called request edge. It means that process  $P_i$  requested an instance of resource type  $R_j$  and is waiting for that resource.
- A directed edge from resource type  $R_j$  to process  $P_i$  is denoted by  $R_j \rightarrow P_i$ , It is called assignment edge. It means that an instance of resource type  $R_j$  has been allocated to process  $P_i$ .

## Resource Allocation Modeling using Graphs



- The sets of
- process  $P = \{ P1, P2, P3 \}$ ,
- resource  $R = \{ R1, R2, R3, R4 \}$ ,
- Edges  $E = \{ P1 \rightarrow R1, P2 \rightarrow R3, R1 \rightarrow P2, R2 \rightarrow P1, R2 \rightarrow P2, R3 \rightarrow P3 \}$
- Resource instances – 1 instance of resource type R1, 2 instances of resource type R2, 1 instance of resource type R3 and 3 instances of resource type R4.
- Here ,Process states –
- Process P1 is holding an instance of resource type R2, and is waiting for an instance of resource type R1.
- Process P2 is holding an instance of resource type R1 and R2, and is waiting for an instance of resource type R3.

- Process P3 is holding an instance of resource type R3.



If a RAG contains no cycles, then no process in the system is deadlocked. If the graph contains a cycle, then deadlock may exist.

## Unit-3 Memory Management

### ❖ Memory Management

- MMU (Memory Management Unit) is a part of OS that deals with main memory.
- MMU is memory manager. Its main goal is to efficiently utilize the main memory among various simultaneously executing processes.
- Requirement and Functionalities of MMU.

It perform operation like

#### 1. Fetch:

- It should determine when to move information from disk to main memory.

#### 2. Placement:

- It should determine where to put fetched information in memory (means memory is allocated for fetched information).

#### 3. Replacement:

- If memory is required but not available it needs to determine which information to remove from memory.

#### 4. Sharing:

- It needs to allow more than one process to share a piece of memory.

#### 5. Address Translation:

- It is responsible for translation logical address to physical address.

#### 6. Protection:

- It protects memory against an unauthorized request for access.
- One process should not have access to unauthorized information of another process.

### ❖ Logical and physical address map

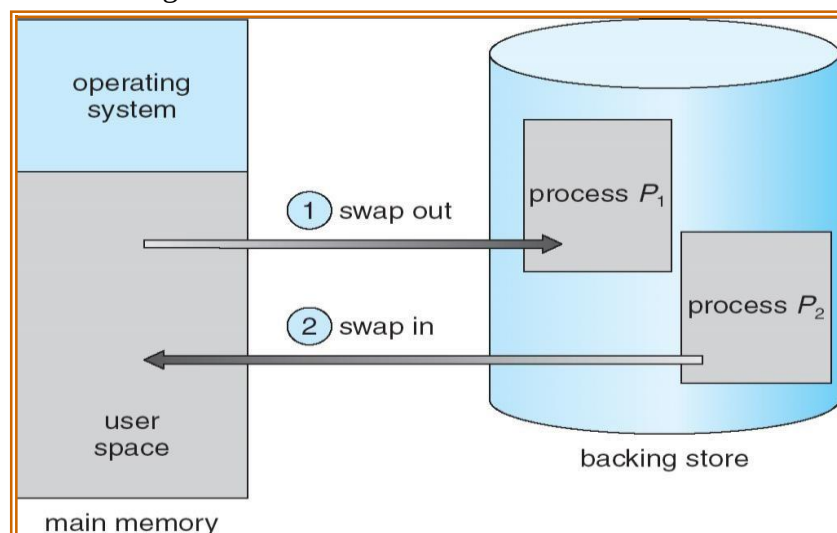
Physical address allocated to the process may not exactly match the logical addresses of process so there should be some mapping between physical address and logical address, so memory manager is required

|    | Logical Address                                                                                                                                                                    | Physical Address                                                                                                                                                                     |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | The process address space can be considered as a sequential list of bytes. Each byte has an address that is used to locate it, these addresses are called <b>logical address</b> . | The entire physical memory can be considered as a sequential list of bytes. Each byte has an address that is used to locate it; these addresses are called <b>physical address</b> . |
| 2. | It is generated by CPU, means CPU determines location of each instruction &                                                                                                        | It is generated by Main Memory.                                                                                                                                                      |

|    |                                                                                                                                       |                                                                                                                |
|----|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
|    | data in process address space.                                                                                                        |                                                                                                                |
| 3. | Logical Address Space is a set of all logical address that can be referenced by a process.                                            | Physical Address Space is a set of physical address occupied by a process in main memory during its execution. |
| 4. | Address in Logical Address Space is a start from 0 and goes up to some maximum value based on size of process                         | Physical Address Space may or may not be contiguous based upon the memory location method.                     |
| 5. | Process can read and write address of its own Logical Address Space.                                                                  | Process can read and write only those physical addresses that belong to its own physical address space.        |
| 6. | Logical Address is limited by address size of the processes.<br><br>Ex:32 – bit processor can generate up to $2^{32}$ (4-GB) address. | Physical Address is limited to amount of memory installed.                                                     |

## ❖ Swapping

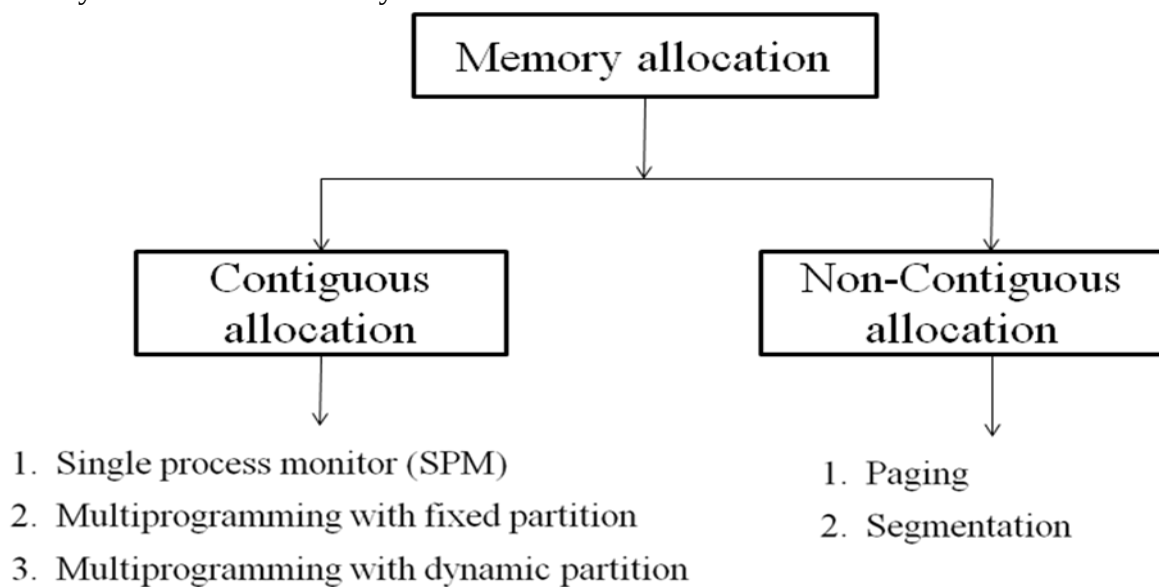
- **Swapping** is a technique in which processes are moved between main memory and disk.
- Swapping uses some of the portion of secondary storage (disk) as a backing store; this area is called **swap area**.
- Operation of moving process from main memory to swap area is **swap-out**.
- Operation of moving processes from swap area to main memory is **swap-in**.
- When a process is brought back to memory, it may be loaded at some different location rather than its original one. So It needs memory relocation,  
There is problem of external fragmentation.



---

## ❖ Memory Allocation

- Memory should be allocated to various processes and files as per requirements.
- When there is no need allocated memory should be freed.
- Memory has a limited capacity compared to disk. So memory should be allocated carefully among all these processes.
- While allocating memory two goals should be fulfilled.
  1. **High Utilization**
    - Maximum possible memory should be utilized.
    - No any single process of memory should be wasted.
  2. **High Concurrency**
    - Maximum processes should be in main memory.
- When more and more processes in main memory, CPU will remain busy most of the time in executing one of the processes, resulting in better throughput.
- Memory is allocated in two ways.

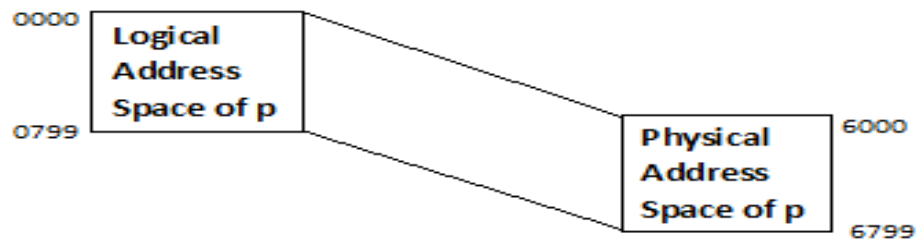


**FIG: Memory Allocation**

### **1. Contiguous Memory Allocation:**

- It is simple and old allocation method.
- It is not used in modern Operating System.
- In this, each processes occupies a block of contiguous memory location in main memory.
- Entire processes kept together in contiguous section of memory.

- When processes is brought in main memory is searched to find a chunk (group) of memory having enough size to hold a processes, once such chunk is found required memory is allocated.
- If contiguous memory space of required size is not available in main memory, this process is made to wait until contiguous space of requires size is available.
- Here logical address space is not divided into any partition.
- Physical Address space will be contiguous without any gaps.

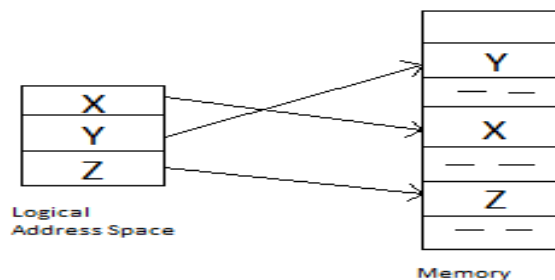


**FIG: Contiguous Memory Allocation**

- Advantage : - Easy to implement and understand.
- Disadvantage: - Having poor memory utilization.

## **2. Non - Contiguous Memory Allocation:**

- This is used by modern Operating System.
- Here logical address space of processes is divided into partition and for each partition contiguous chunk of free memory is allocated.
- Physical Address space will not be contiguous now.



**FIG: Non - Contiguous Memory Allocation**

- Advantage: - Having better memory utilization.
- Disadvantage: - Complex to implement and understand.

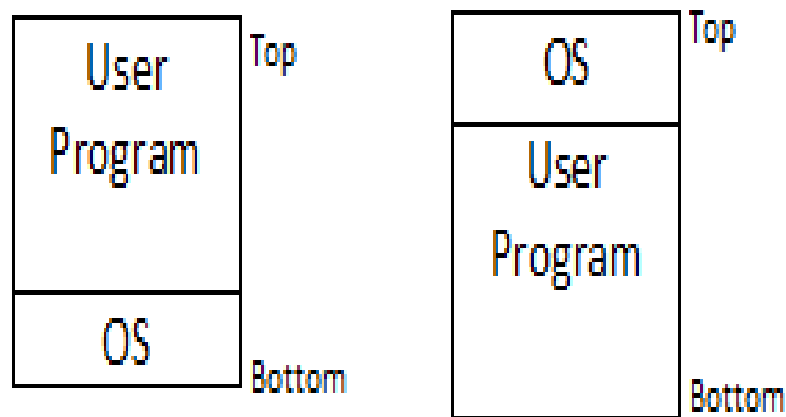
---

## ❖ Contiguous memory allocation

- Three different ways for contiguous memory allocation:
  1. Single Process Monitor (SPM)
  2. Multiprogramming with Fixed partition.(static partition)
  3. Multiprogramming with Dynamic partition. (variable partition)

### Single Process Monitor(SPM):

- It is simplest possible memory management scheme.
- Only one process is allowed to run.
- No more than one process can run simultaneously.
- Memory is shared between process and Operating System.



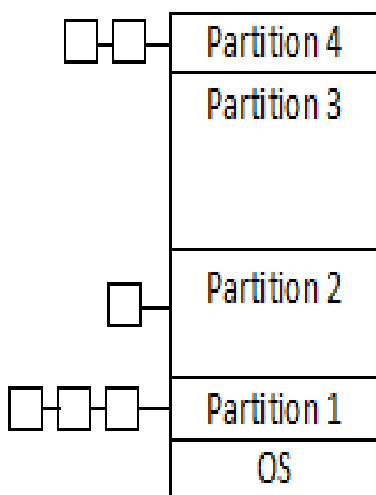
**FIG: Single Process Monitor**

- This method can be used in palmtop computer, Embedded System, pc running only in MS-DOS :
- Only one process can run at a time
- When user types a command the Operating System copies requested program from disk to memory and executes it.
- When process finishes, the Operating System displays a prompt character and waits for a new command.
- When Operating System loads new program in main memory, overwriting first one and execute it.
- Advantage: -
  - Simplicity.
- Disadvantage: -
  - Only one process can be executed at a time.

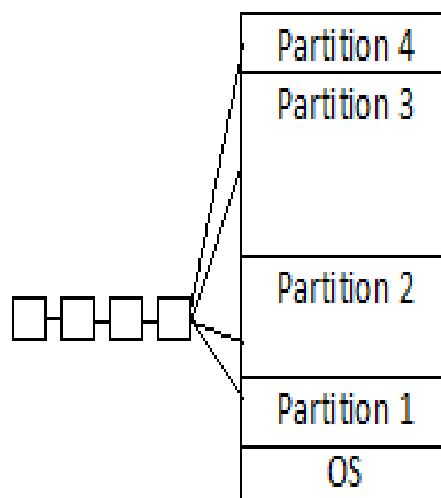
---

### ❖ Multiprogramming with fixed (static) partition:

- This method allows multiple processes to execute simultaneously.
- Memory is shared among Operating System and various simultaneously running processes.
- Multiprogramming increases the CPU utilization, CPU can be kept busy almost all time by keeping more than one processes simultaneously in memory.
- Memory is divided into fixed partition size can be equal or unequal for different partition. Generally unequal partitions are used for better utilization of memory.
- Each partition is accommodating exactly one process.
- Whenever program needs to be loaded in memory, a free partition big enough to hold program is found and allocated.
- If there is no free partition available of required size, which process needs to wait such process will be put in a queue.
- There are two possible ways to implement method with a queue.
  1. Using multiple input queues. – Fig (a)
  2. Using single input queue. – Fig (b)



Fig(a)



Fig(b)

---

## 1. Using Multiple Input Queues.

- For each partition separate queue is maintained.
- Whenever any process comes, it is put in a queue for smallest partition large enough to hold it.
- When partition becomes free a process from the front end of the queue is allocated that partition.
- Disadvantage: - If queue for a large partition is empty, but for small partition is full, small process needs to wait to get memory.
- Here free memory is available but it cannot be used for small process.

## 2. Using Single Input Queue.

- Only single queue is maintained for all partition.
- Whenever any partition become free, process from queue is selected, this can be hold by that partition.
- Process can be selected in two ways
- Just select process which is near to front of the queue.
  - Disadvantage: large partition will be wasted for small process.
- Search entire queue and select largest process which can fit in that partition.
  - Disadvantage: Starvation possible for small process.

### Advantage of fixed partition method:

1. Implement is simple.
2. processing overhead are low.

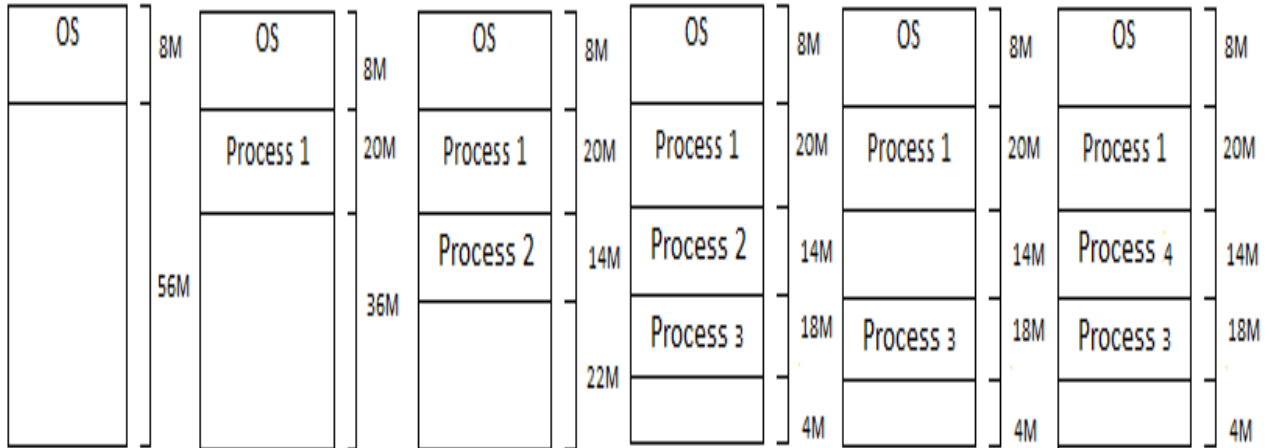
### Disadvantage of fixed partition method:

1. Degree of multiprogramming is fixed here, the maximum number of process can be executed simultaneously can't be changed.
2. Partitions are of fixed size, so any space in a partition not used by a process is wasted, this is called **Internal Fragmentation**.

### ❖ Multiprogramming with dynamic partition:

- This method allows multiple processes to execute simultaneously.
- Here memory is shared among Operating System and various simultaneously running processes.
- Here **memory is not divided into any fixed size partition. And number of partition is not fixed, process is allocated exactly as much memory as it requires.**
- Initially entire available memory is treated as a single free partition.
- Whenever any process enters in a system a chunk of free memory big enough to fit the process is found and allocated.

- If enough free memory is not available to fit the process, process needs to wait until required memory becomes available.
- Whenever any process gets terminated, it releases the space occupied. If the released space is contiguous to another free partition, they are clubbed together into a single free partition.



**FIG: Multiprogramming with dynamic partition**

- Advantage:
  1. Better Utilization of memory
  2. **No internal fragmentation** because process gets only that much memory of its size.
  3. Degree of multiprogramming (means no. of processes running at same time in memory) is not fixed here.
- Disadvantage:
  1. Memory is allocated when process enters in the system and released when it terminates. These operations externally result in small holes in the memory. These holes will be small that no any process can be loaded in it, but total size of all holes may be big enough to hold any process. But memory is allocated contiguous here holes can't be used this type of memory wastage is called **External Fragmentation**
  2. Scattered (spread) free memory partition can be combined using compaction or De-Fragmentation.

### ❖ **Fragmentation (wastage of memory):**

- Memory is allocated when process enters in the system and released when it terminates.
- We can't utilize (100%) full memory due to some problems (fragmentation).
- **Fragmentation refers to unused memory that cannot be allocated to any process.** Means there is free memory available but it can't be used.
- Two kind of fragmentations:
  1. External Fragmentation

---

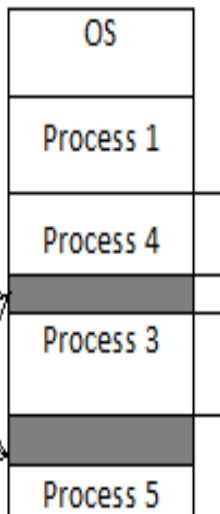
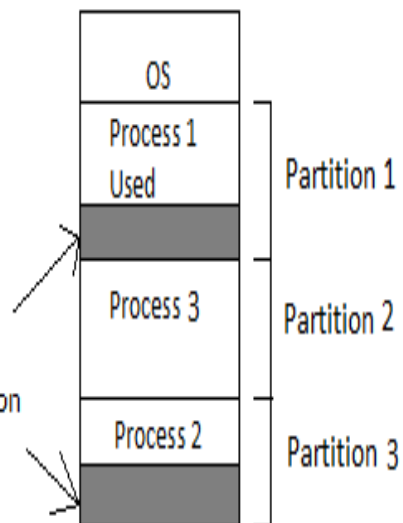
## 2. Internal Fragmentation

### ❖ Internal and External Fragmentation and compaction

|    | <b>External Fragmentation</b>                                                                                                                                                                                                   | <b>Internal Fragmentation</b>                                                                                                                                        |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | It refers to wastage of memory between partitions.                                                                                                                                                                              | It refers to wastage of free memory within partitions.                                                                                                               |
| 2. | It is caused by scattered non – contiguous free space.                                                                                                                                                                          | It is caused by the differences between the size of partition and size of process loaded.                                                                            |
| 3. | External Fragmentation is a severe problem in a contiguous memory allocation method with dynamic partitioning.                                                                                                                  | Internal Fragmentation is a severe problem in a contiguous memory allocation method with fixed partition.                                                            |
| 4. | Memory allocation & de-allocation operations eventually result in small holes in memory. These holes will be so small that no any processes can be loaded in it, but total size of all holes may be big enough to hold process. | When memory de-allocates whole partition may be free, so it can be allocated other process. So there will not any hole in partition. operations eventually result in |
| 5. | As memory is allocated contiguously, here the holes can't be used. This type of memory wastage is called <b>External Fragmentation</b> .                                                                                        | Partitions are of fixed size. So any space in a partition not used by a process is wasted this is <b>Internal Fragmentation</b> .                                    |
| 6. | Solution: Compaction or de-fragmentation of memory.                                                                                                                                                                             | Solution: To use dynamic partitioning where process is allocated exactly as much memory as possible.                                                                 |

❖  
❖  
❖  
❖  
❖

7.

External  
FragmentationInternal  
Fragmentation

## ❖ Memory relocation and protection mechanism

### 1. Memory Relocation:

- A process can loaded in any partition in main memory.
- Address in Logical Address Space and Physical Address Space is not a same here.
- Logical Address Space specifies the location of instructions and data within process address space.
- Physical Address Space specifies actual location in main memory.
- Logical Address is required to actually fetch instruction. So whenever there is a reference to any Logical Address it should be converted to physical address this problem is called memory relocation.
- EX: suppose process is loaded at location 1000 in main memory & there is need to fetch instruction located at location 5 in Logical Address Space.
- So Logical Address (5) should be converted to actual Physical Address  $(1000 + 5) = 1005$ .

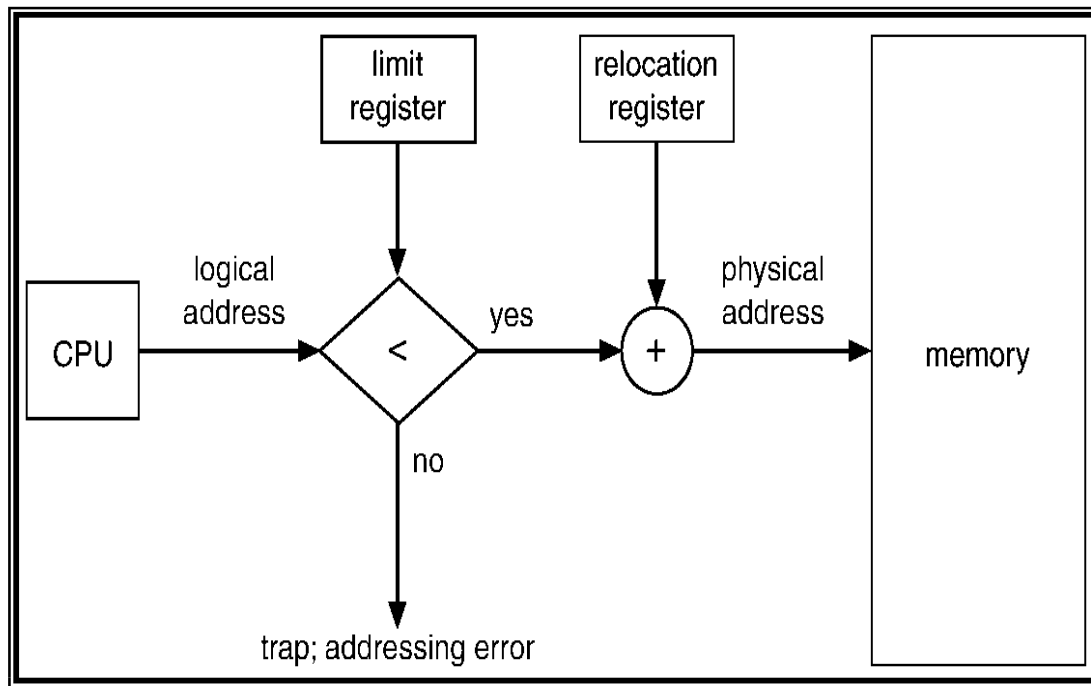
### 2. Memory Protection:

- Multiprogramming allows running more than one process run simultaneously.
- So memory is shared among processes as well as Operating System.
- All concurrent processes will be in the main memory at the same time, these processes should not have accessed to unauthorized of other processes. Process should read and write data belonging to that process only. This problem is called memory protection.

#### Solution to both the problem:-

- Two register are used. These registers are called **limit register** and **base register**.
- Limit register are used to store size of process
- Base register are used to store starting location of process in main memory.

- Whenever any process is loaded in main memory, its starting location and size are stored in these two registers respectively.
- CPU generates logical address start from 0 and goes upto size of process.
- Thus in contiguous memory allocation, the problems of memory relocation and protection can be overcome by maintaining two registers limit register and base register



**FIG: Memory Relocation & Protection**

## ❖ Allocation techniques – First Fit, Best Fit and Worst Fit

### Strategies to select partition:-

- Whenever any process enters in a system required free memory is allocated for that process.
  - All free memory, also called holes, is searched to find out chunk of required free memory.
  - Four strategies or algorithm are to do this
- 1. First Fit:**
    - Search start from starting location of memory.
    - First available hole, which is large enough to hold the processes, is selected for allocation.
    - Search time is small here.
    - Memory loss is higher as very large hole may be selected for small process.
  - 2. Next Fit:**
    - This is a minor variation over first fit.
    - Searches start from where previous search ended.

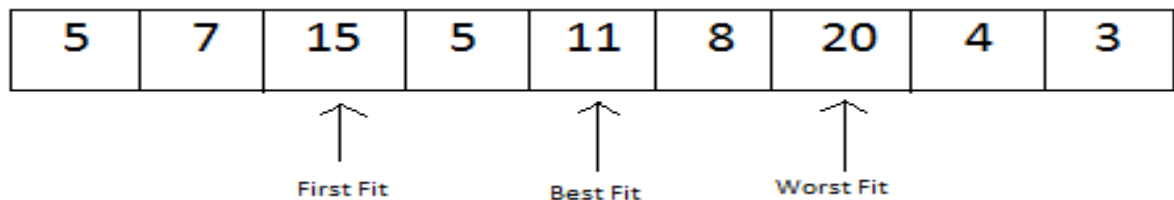
- This is avoiding repeating search of memory which has been already allocated; such partition will be mostly at front end of the main memory.

### 3. **Best Fit:**

- Entire memory is searched here.
- The smallest hole, which is large enough to hold the processes, is selected for allocation.
- Search time is high, as it searches entire memory.
- Memory loss is less, more sensitive to external fragmentation, as it leaves tiny holes into which no process can fit.

### 4. **Worst Fit:**

- Entire memory is searched here.
- The largest hole, which is large enough to hold the processes, is selected for allocation.
- It is not sensitive to external fragmentation.
- This algorithm can be used only with dynamic partition.



## ❖ **Non Contiguous Memory allocation**

There are two types of non-contiguous allocation techniques:

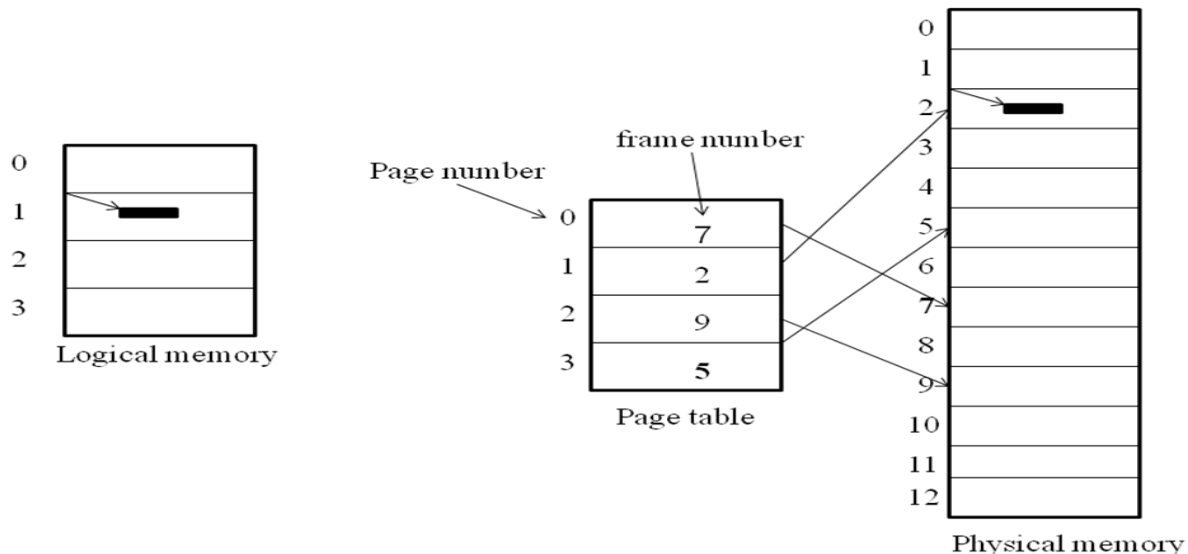
1. **Paging**
2. **Segmentation**

### ❖ **Paging Technique**

- Logical address space of a process is divided into blocks of fixed size, called **pages**.
- Physical memory is divided into blocks fixed size; called **frames** (page frames).
- The page and frame will be of the same size. This size will be power of 2. It varies between 512 bytes to a few MB.
- **Whenever a process is to be executed, its pages are moved from secondary storage (disk), to the available frames in physical memory.**
- Memory allocation refers to,
  - The task of finding free frames in memory,
  - allocating them to pages, and
  - Keeping track of which frame belongs to which page.

- **Page table**

- Operating system maintains a table, called **page table** for each process.
- It is indexed by a **page number**.
- Page table contains information about frame no. in which pages are stored.



**Fig -1: Basic concept of paging**

Here, the logical address space of a process, also called logical memory, has been divided into four pages. Physical memory contains total of 11 frames. Also, a page table is given which shows the mapping between pages and frames.

### Implementation of Paging technique

- Here, logical address is divided into two parts:
  - **page number**, which gives the number of a page
  - **An offset**, which gives the actual location within a page.

**Logical address ( L ) :**

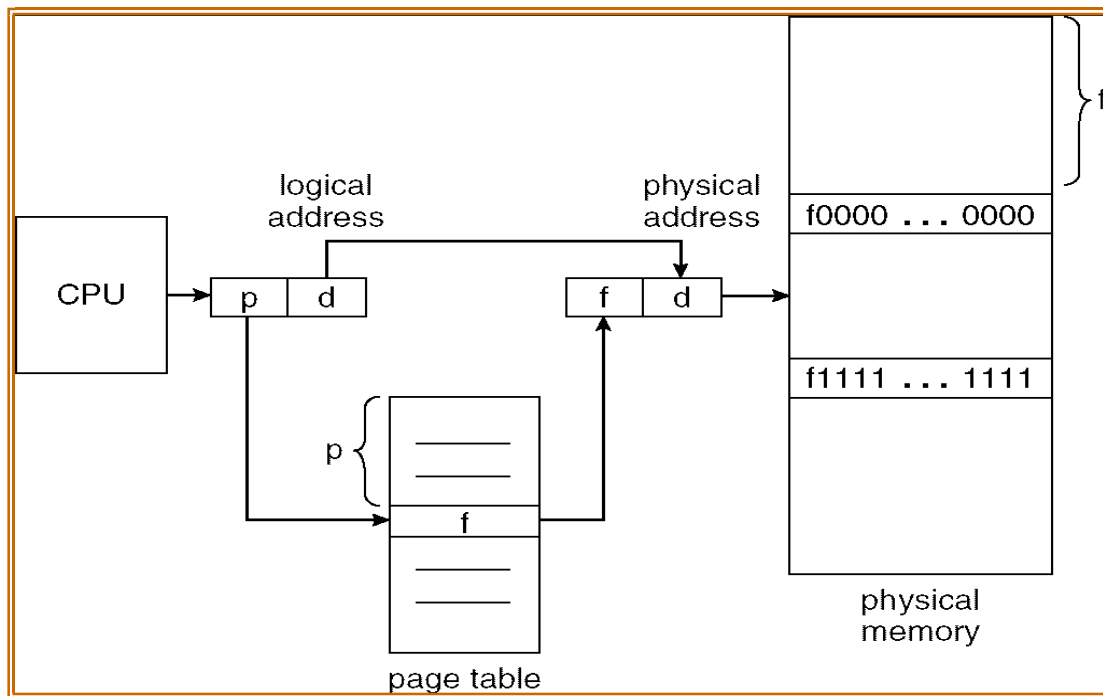
|                   |                  |
|-------------------|------------------|
| <b>Page no(p)</b> | <b>offset(d)</b> |
|-------------------|------------------|

- **Page table is used to implement paging.**
- When process is to be executed, pages (logical memory) **are moved to free frames** (main or physical memory).
- During the process execution ,a Logical address (L) is generated by CPU during process execution , to access instruction or data from page table
- Page number (**p**) is used to search any page from page table and if that page is searched then related frame number (**f**) is obtained from page table.
- Frame no. indicates actual frame on physical memory in which page is stored.

**Physical address (P):**

|                    |                  |
|--------------------|------------------|
| <b>frame no(f)</b> | <b>offset(d)</b> |
|--------------------|------------------|

Following figure explains paging implementation



**Fig-2: Address translation in paging**

**Address translation:**

1) Page size and frame size =  $2^n$  bytes.

2) **Logical address (L) :**

a. **Page no (p) :**  $L / 2^n$

b. **offset (d) :**  $L \% 2^n$

3) **physical address (P) :**  $f * 2^n + d$

**Example :** ( No need to give example in GTU exam if not asked specifically)

If page size (or frame size) is 4 byte ( $2^2$  byte). The logical address generated by the CPU is 11. then calculate physical address.

**Ans :** page size=frame size =  $2^2$  byte

Logical address (L) :

1) page no (p) =  $L / 2^2 = 11 / 4 = 2$

2) offset (d) =  $L \% 2^2 = 11 \% 4 = 3$

from fig-1 ,we can get frame no 9 for page no. 2

so now **physical address (P) :**  $f * 2^{n+d}$

$$= 9 * 4 + 3 = 39$$

**Advantage of paging :**

1. Allocation and de-allocation of process is very fast .only list of free frames need to be maintained .No any page algorithm is needed.
2. Swap-in and swap-out operations are very fast.(because page and frame size are same ,data moves easily )

- 
3. Here, available memory is divided into fixed sized block and no wastage of memory between partitions, so there is no External fragmentation.

**Disadvantage of paging:**

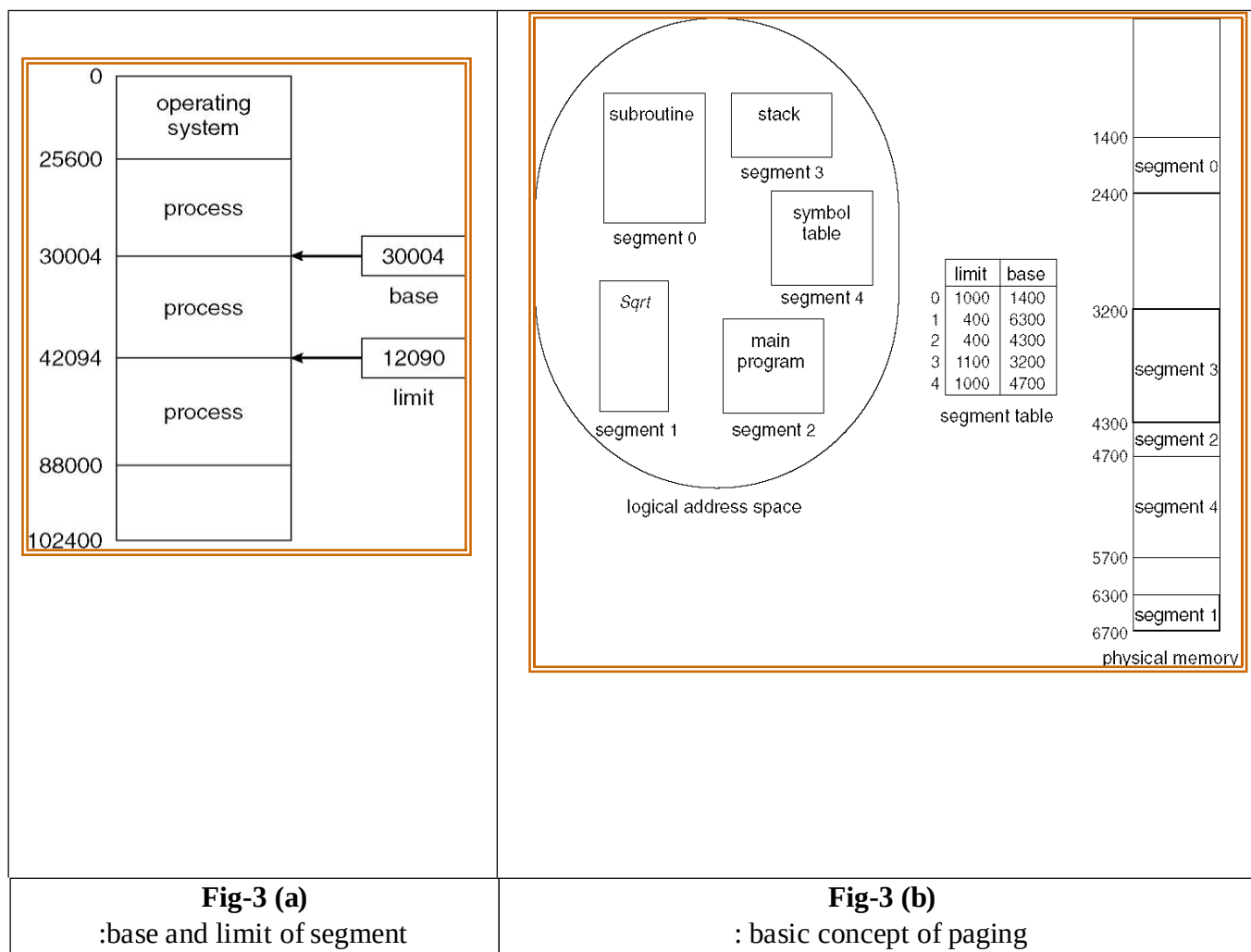
1. Additional memory reference is required to read information from page table .Two memory access required one for page table and second for instruction or data.
2. If process is of large size then page table of that process also have large size. It may be too large to keep such page table in memory.
3. Here, it may be possible that space within partitions (page) remain unused by process because of differ in page size and part of process loaded in that page . So paging suffers from internal fragmentation.

❖ **Segmentation**

- Logical address is collection of code, data and stack. **Code** can be main function, user define function library function. **Data** can be local and global variable, array, symbol table or other data structure.
- Logical address space of a process is divided into blocks of varying size, called **segment**. Each segment contains a logical unit of a process.
- Logical unit can be main function, user define function, procedure, stack, array, symbol table.
- Each segment can be considered as a independent address space.
- Segment consists of linear sequence of address starting from '0' to some maximum limit. All segments are of varying lengths and length depends upon the size of a logical unit
- All segments are given unique numbers to identify them.
- When process is to be executed its segments are moved from secondary storage to main memory.
- OS maintains **segment table** for each process. **Segment table** contains two information about each process.
  1. **Size of segment**
  2. **Offset** , which gives actual location within segment

**Implementation of segment:**

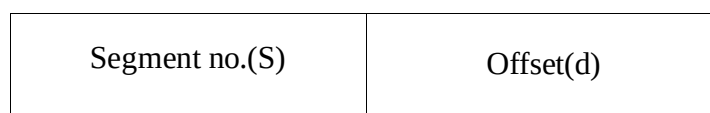
- Segment table is used to implement segmentation .Each entry of the segment table has a segment **base** and segment **limit**.
  - A segment **base** contains the starting physical address where the segment resides in memory.
  - The segment **limit** specifies length of segment.



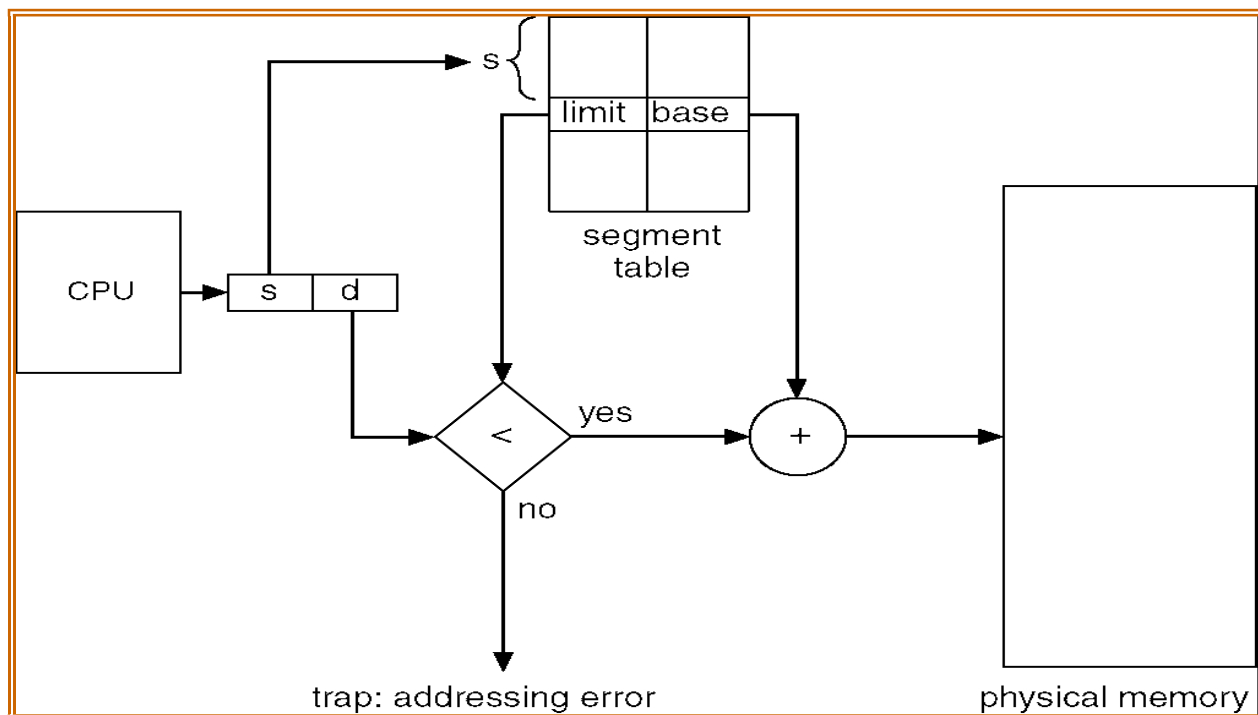
- A pair of **base** and **limit** registers define the logical address space and stores segment base and segment limit.

Logical address:

- During the process execution ,a CPU generates a logical address (L) to access instruction or data from a particular location.
- Logical address is divided into two parts as shown in figure:



- The segment number is used as index into segment table. The offset (d) of the logical address must be between 0 and the segment limit.
- If offset is within range of limit register value than offset is added to the segment base to produce the physical address.
- If offset is not within limit than illegal address error will be generated.



#### Example : (refer Fig-3 (b))

- Suppose CPU has generates some logical address, and is given form of(segment no , offset )-  
L=(2,125)
- To get physical address related to logical address, segment table will be searched to get entry of segment no 2.
  - For segment 2 ,**base is 4300** and **limit is 400**,
  - **Offset compared with limit value ( 125 < 400)**

So, offset is valid

$$\text{Physical address (P)} = \text{offset} + \text{base}$$

$$= 125 + 4300 = 4425$$

#### Example 2

- Suppose CPU has generates some logical address, and is given form of(segment no , offset )-  
L=(2,410)
- To get physical address related to logical address, segment table will be searched to get entry of segment no 2.
  - For segment 2 ,**base is 4300** and **limit is 410**,
  - **Offset compared with limit value ( 125 < 410)**

So, offset is not valid. so physical address can not be generated.

#### Advantage:

- All segments are independent from each other, so segments can grow and shrink without affecting other segments.
- If procedure in one segment is modified and recompiled, no other segments need to be changed or recompiled.

- 
- Sharing of procedure and data between various processes is simple.
  - Different segment of a single process can be given different kind of protection. One segment can be read only ,while another can be writable.
  - No internal fragmentation because segments are allocated exactly as much memory as required.

**Disadvantage:**

- It is still expensive to allocate contiguous free memory to segments.
- External fragmentation is possible, which requires memory de-fragmentation or compaction.

❖ **Page replacement algorithm – FIFO, LRU**

There are various page replacement algorithms. Each algorithm has a different method by which the pages can be replaced.

1. **Optimal Page Replacement algorithm** → this algorithms replaces the page which will not be referred for so long in future. Although it cannot be practically implementable but it can be used as a benchmark. Other algorithms are compared to this in terms of optimality.
2. **Least recent used (LRU) page replacement algorithm** → this algorithm replaces the page which has not been referred for a long time. This algorithm is just opposite to the optimal page replacement algorithm. In this, we look at the past instead of staring at future.
3. **FIFO** → in this algorithm, a queue is maintained. The page which is assigned the frame first will be replaced first. In other words, the page which resides at the rare end of the queue will be replaced on the every page fault.

Consider a main memory with five page frames and the following sequence of page references: 3, 8, 2, 3, 9, 1, 6, 3, 8, 9, 3, 6, 2, 1, 3. which one of the following is true with respect to page replacement policies First-In-First-out (FIFO) and Least Recently Used (LRU)?

- A. Both incur the same number of page faults
- B. FIFO incurs 2 more page faults than LRU
- C. C. LRU incurs 2 more page faults than FIFO
- D. D. FIFO incurs 1 more page faults than LRU

Solution:

Number of frames = 5

FIFO

According to FIFO, the page which first comes in the memory will first goes out.

|          |      |      |      |     |      |      |      |      |      |     |     |     |      |     |     |
|----------|------|------|------|-----|------|------|------|------|------|-----|-----|-----|------|-----|-----|
| Request  | 3    | 8    | 2    | 3   | 9    | 1    | 6    | 3    | 8    | 9   | 3   | 6   | 2    | 1   | 3   |
| Frame 5  |      |      |      |     |      | 1    | 1    | 1    | 1    | 1   | 1   | 1   | 1    | 1   | 1   |
| Frame 4  |      |      |      |     | 9    | 9    | 9    | 9    | 9    | 9   | 9   | 9   | 2    | 2   | 2   |
| Frame 3  |      |      | 2    | 2   | 2    | 2    | 2    | 2    | 8    | 8   | 8   | 8   | 8    | 8   | 8   |
| Frame 2  |      | 8    | 8    | 8   | 8    | 8    | 8    | 3    | 3    | 3   | 3   | 3   | 3    | 3   | 3   |
| Frame 1  | 3    | 3    | 3    | 3   | 3    | 3    | 6    | 6    | 6    | 6   | 6   | 6   | 6    | 6   | 6   |
| Miss/Hit | Miss | Miss | Miss | Hit | Miss | Miss | Miss | Miss | Miss | Hit | Hit | Hit | Miss | Hit | Hit |

Number of Page Faults = 9

Number of hits = 6

### LRU

According to LRU, the page which has not been requested for a long time will get replaced with the new one.

|          |      |      |      |     |      |      |     |     |      |     |      |     |      |      |     |
|----------|------|------|------|-----|------|------|-----|-----|------|-----|------|-----|------|------|-----|
| Request  | 3    | 8    | 2    | 3   | 9    | 1    | 6   | 3   | 8    | 9   | 3    | 6   | 2    | 1    | 3   |
| Frame 5  |      |      |      |     |      | 1    | 1   | 1   | 1    | 1   | 1    | 1   | 2    | 2    | 2   |
| Frame 4  |      |      |      |     | 9    | 9    | 9   | 9   | 9    | 9   | 9    | 9   | 9    | 9    | 9   |
| Frame 3  |      |      | 2    | 2   | 2    | 2    | 2   | 2   | 8    | 8   | 8    | 8   | 8    | 1    | 1   |
| Frame 2  |      | 8    | 8    | 8   | 8    | 8    | 6   | 6   | 6    | 6   | 6    | 6   | 6    | 6    | 6   |
| Frame 1  | 3    | 3    | 3    | 3   | 3    | 3    | 3   | 3   | 3    | 3   | 3    | 3   | 3    | 3    | 3   |
| Miss/Hit | Miss | Miss | Miss | Hit | Miss | Miss | Hit | Hit | Miss | Hit | Miss | Hit | Miss | Miss | Hit |

Number of Page Faults = 9

Number of Hits = 6

---

Consider a reference string: 4, 7, 6, 1, 7, 6, 1, 2, 7, 2. the number of frames in the memory is 3. Find out the number of page faults respective to:

1. FIFO Page Replacement Algorithm
2. LRU Page Replacement Algorithm

### LRU Page Replacement Algorithm

|          |      |      |      |      |     |     |     |      |      |     |
|----------|------|------|------|------|-----|-----|-----|------|------|-----|
| Request  | 4    | 7    | 6    | 1    | 7   | 6   | 1   | 2    | 7    | 2   |
| Frame 3  |      |      | 6    | 6    | 6   | 6   | 6   | 6    | 7    | 7   |
| Frame 2  |      | 7    | 7    | 7    | 7   | 7   | 7   | 2    | 2    | 2   |
| Frame 1  | 4    | 4    | 4    | 1    | 1   | 1   | 1   | 1    | 1    | 1   |
| Miss/Hit | Miss | Miss | Miss | Miss | Hit | Hit | Hit | Miss | Miss | Hit |

Number of Page Faults in LRU = 6

### FIFO Page Replacement Algorithm

|          |      |      |      |      |     |     |     |      |      |     |
|----------|------|------|------|------|-----|-----|-----|------|------|-----|
| Request  | 4    | 7    | 6    | 1    | 7   | 6   | 1   | 2    | 7    | 2   |
| Frame 3  |      |      | 6    | 6    | 6   | 6   | 6   | 6    | 7    | 7   |
| Frame 2  |      | 7    | 7    | 7    | 7   | 7   | 7   | 2    | 2    | 2   |
| Frame 1  | 4    | 4    | 4    | 1    | 1   | 1   | 1   | 1    | 1    | 1   |
| Miss/Hit | Miss | Miss | Miss | Miss | Hit | Hit | Hit | Miss | Miss | Hit |

Number of Page Faults in FIFO = 6

## **Unit-4 File Management**

### **❖ Files System**

It is a part of OS which deals with files.

It manages all files, provides facilities to create, delete, read and write operation for files.

It provides directories to organize files.

It is responsibility of file system

To provide efficient access to information stored in files

To manage memory space on secondary storage devices when file is created, deleted or modified.

To provide protection mechanism to allow users to administer how other users can access the information in files.

### **❖ Files Attributes**

Every file has its name and data. All file contains some other properties called attributes. Main attributes are as following:

#### **1. Name :**

- A string of alphanumeric characters and some special characters like underscore ( \_ ).
- We can refer file easily with its name.

#### **2. Identifier:**

- It is unique tag number which uniquely identifies file within the system.
- It is used by OS to refer files.

#### **3. Type :**

- It identifies type of file.
- It expressed generally in form of extension like **.c , .cpp , .txt**

#### **4. Location:**

- It is the pointer to the device and location on that devices of the file

#### **5. size :**

- It specifies current size of files (in bytes, words or blocks).

#### **6. Protection:**

- This attribute defines who can read file, write files, executes files.

#### **7. Usage count:**

- It indicates no. of processes that are currently using these files.

#### **8. Time , date and user identification:**

- It specifies information regarding file creation, update and last access.

### **❖ File Operations**

User can retrieve file information anytime. OS performs various operations on files.

OS provides a set of system call to perform such operations.

This system calls can be called using library functions provided by programming languages.

Basic file operations are as follows:

**1. Create:**

- A new file can be created by system call embedded in a program or by an OS commands issued by an interactive users.
- To create file two steps are performed; first required disk space is allocated to file; second, new entry for the created file is made in directory.
- **The directory entry contains file name, identification number and some other information.**

**2. Delete:**

- When file is not needed, it has been deleted to free up disk space.
- Two steps for deletion:
  - 1. All the space on disk is released,
  - 2. Directory entry is erased.

**3. Open:**

- To use file, it must be opened.
- File attributes and data contents are fetched in main memory for fast access on later calls.
- Memory space in main memory is allocated to store fetched information.

**4. Close:**

- When use of file is finished, it should be closed to free up main memory space.
- File attributes and contents are stored back on disk. This may contain modified information if file is updated.

**5. Read:** To Read the information which is Stored into the Files.

**6. Write:** For inserting some new Contents into a File.

**7. Append:** user can add data at the end of existing file.

**8. Seek:** if user wants to read random file data, then using this operation user can skip some lines and go forward to read.

**9. get attributes :** user can get attribute information

**10. set attribute :** user can set attribute information

**11. rename:** user can rename the file name.

## ❖ File Types

### Regular files:

Regular File may belong to any type of Application for example notepad, paint, C Program, Songs etc.

- Ordinary Files are used for Storing the information about the user Programs. With the help of Ordinary Files we can store the information which contains text, database, any image or any other type of information.

### Directory files:

The Files those are Stored into the Particular Directory or Folder. Then these are the Directory Files. Because they belongs to a Directory and they are Stored into a Directory or Folder. For Example a Folder Name Songs which Contains Many Songs So that all the Files of Songs are known as Directory Files.

### **Special Files. :**

The Special Files are those which are not created by the user. Or The Files those are necessary to run a System. The Files those are created by the System. Means all the Files of an Operating System or Window, are refers to Special Files. There are Many Types of Special Files, System Files, or windows Files, Input output Files. All the System Files are Stored into the System by using. sys Extension.

## **❖ Directory Structures**

### **1. Single level directory structure**

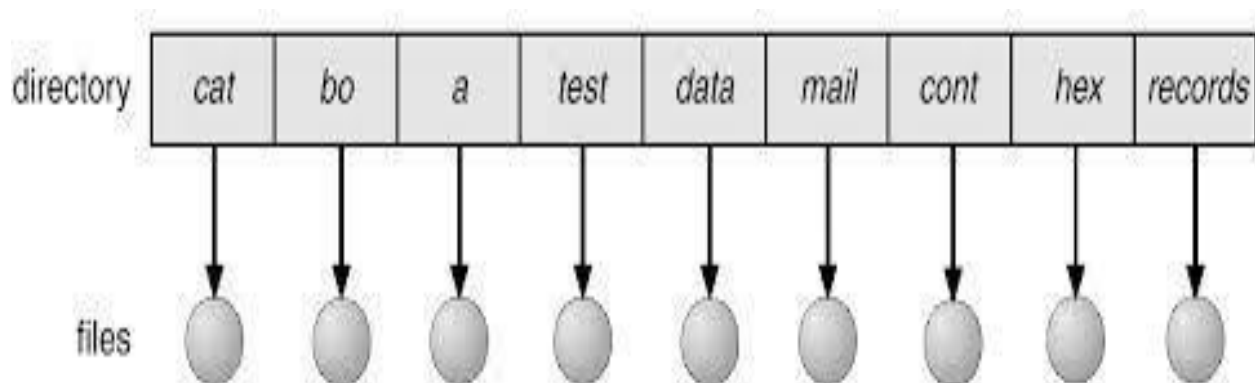
- It is simplest directory structure.
- In this type of structure all files are stored in same directory.
- Directory Contains only files, no sub directory.
- It was Used in Pc

#### **Advantage**

- Simple
- Easy to search

#### **Disadvantage**

- Not suitable for multiuser system
- No. of files becomes too large
- Different files can't be grouped
- Since all files are in the same directory, they must have unique name.
- Single user may find it difficult to remember the names of all files as the number of file increases.
- Keeping track of so many file is tedious task.



## 2. Two level directory structure

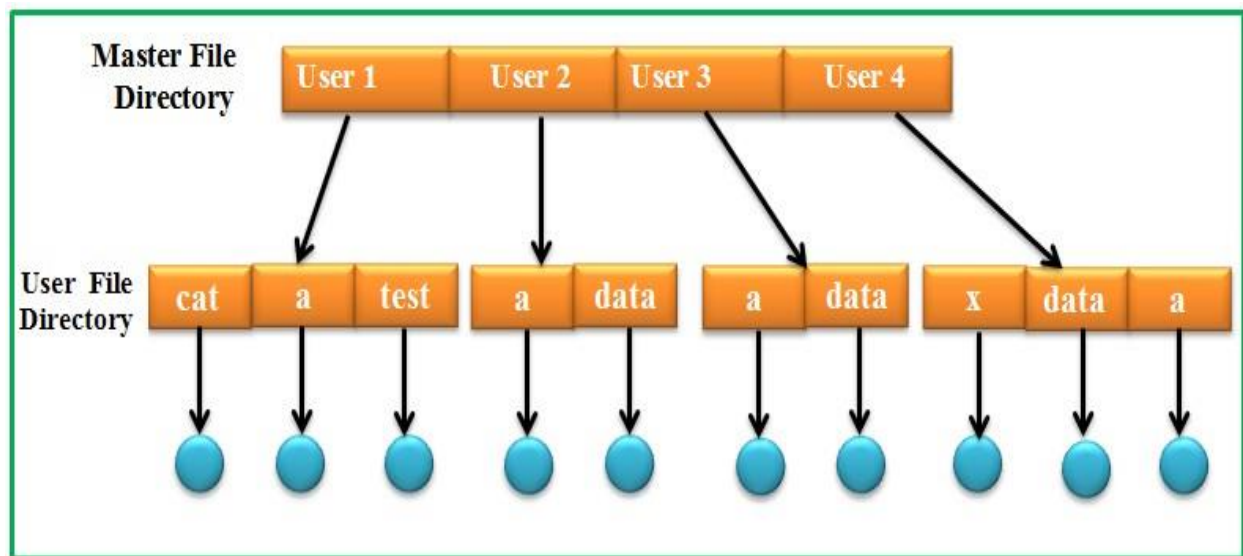
- Each user has private directory to avoid file collision
- Two level directory
  - Root directory(master file directory **MFD**)
  - User file directory(**UFD**)
- The root of a tree is Master File Directory (MFD). Its subdirectory is User File Directory (UFD).
- The descendents of UFD's are file themselves

### Advantage

- Avoid file collision
- Used in multi user system
- Efficient searching

### Disadvantage

- Not suitable for user having large no. of file
- Different files cannot be grouped.

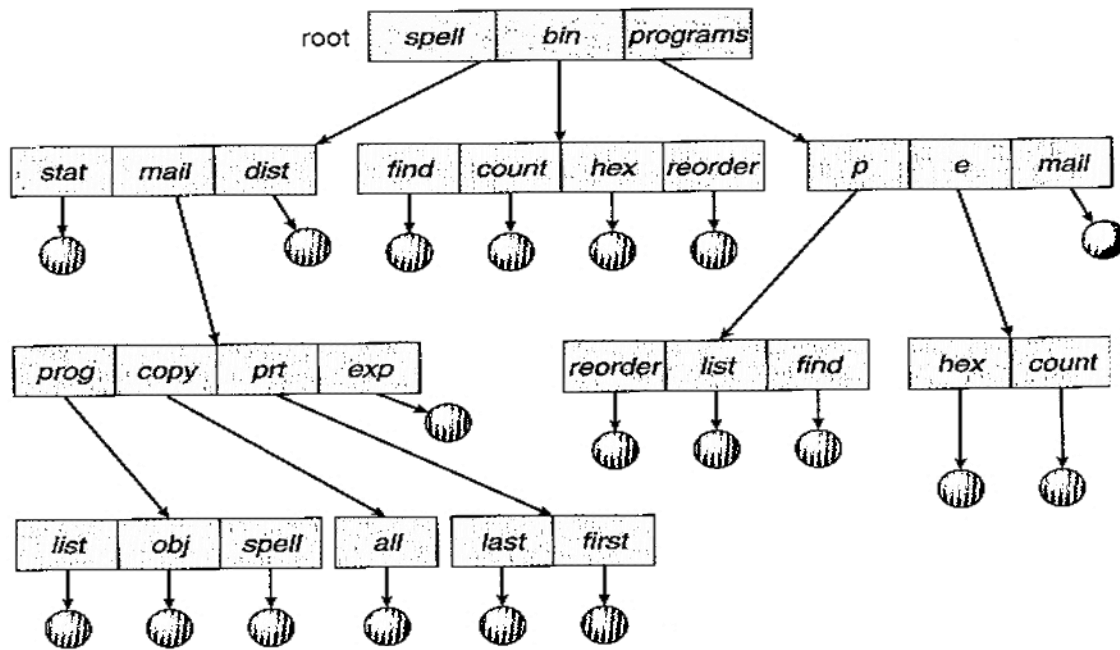


## 3. Tree structured directory structure

- User can create their own sub directory
- It is also Known as hierarchical system
- Each file has unique file path.
- Advantage
  - It Avoid file collision
  - It is Used in multi user system
  - Different files can be grouped

- **Disadvantage**

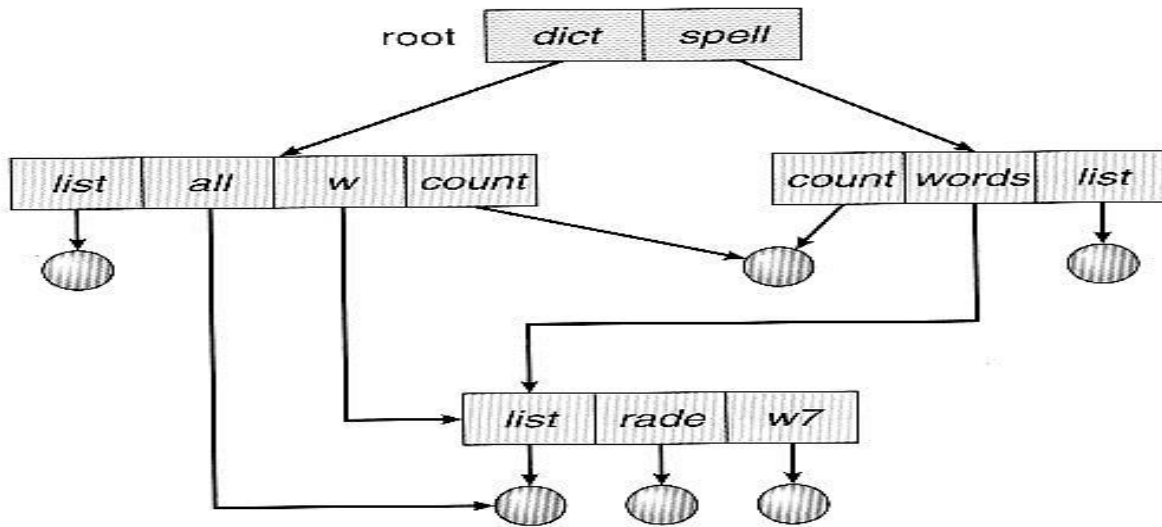
- Sharing of files and directory is not possible.



**Figure 10.9** Tree-structured directory structure.

#### 4. Acyclic graph directory structure

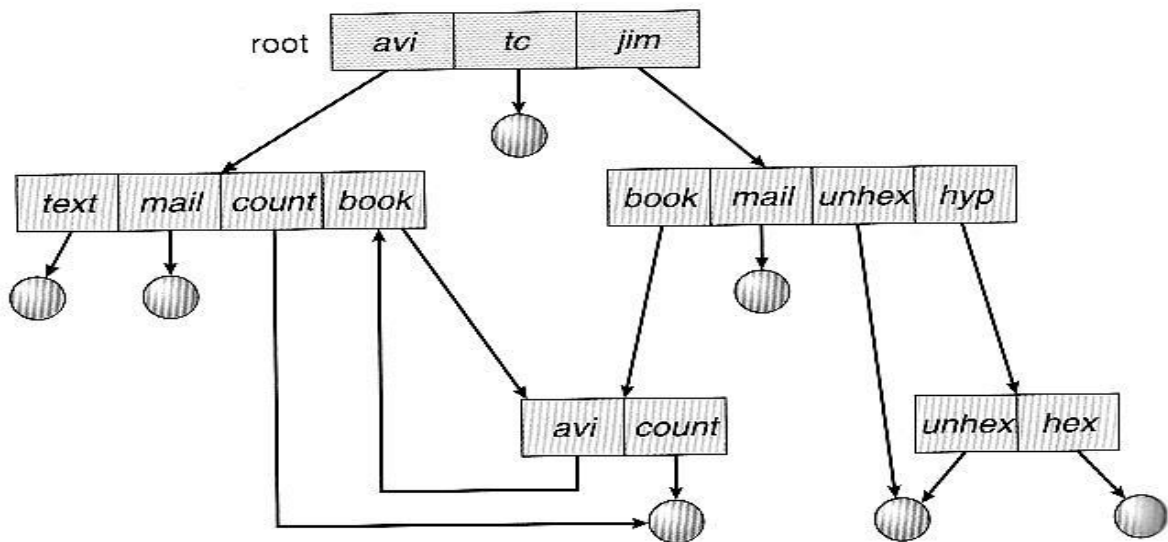
- When the same files need to be accessed in more than one place in the directory structure ( e.g. because they are being shared by more than one user / process ), it can be useful to provide an acyclic-graph structure. ( Note the **directed** arcs from parent to child. )
- Permits user to create shared files and director
- shared files and directory created using links
- Directory structure does not contain cycle.
- **Advantage**
  - Sharing of files and directory
- **Disadvantage**
  - Deletion of files or dir. Is complex.
  - There are no cycle in directory
  - File may have multipath



**Figure 10.10** Acyclic-graph directory structure.

### 5. General graph\_directory structure

- Same as acyclic graph but allows cycles in directory
- **Advantage**
  - Allows sharing of file
- **Disadvantage**
  - Searching should be done careful ,it may get infinite.



**Figure 10.11** General graph directory.

**❖ Protection**

Protection is especially important in a multiuser environment when multiple users use computer resources such as CPU, memory, etc. It is the operating system's responsibility to offer a mechanism that protects each process from other processes. In a multiuser environment, all assets that require protection are classified as objects, and those that wish to access these objects are referred to as subjects. The operating system grants different 'access rights' to different subjects.

A mechanism that controls the access of programs, processes, or users to the resources defined by a computer system is referred to as protection. You may utilize protection as a tool for multi-programming operating systems, allowing multiple users to safely share a common logical namespace, including a directory or files.

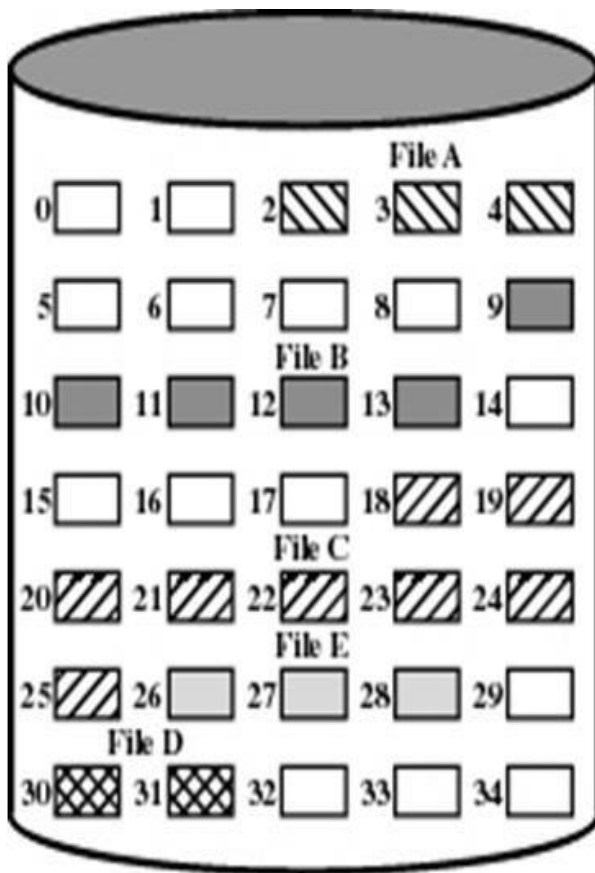
It needs the protection of computer resources like the software, memory, processor, etc. Users should take protective measures as a helper to multiprogramming OS so that multiple users may safely use a common logical namespace like a directory or data. Protection may be achieved by maintaining confidentiality, honesty and availability in the OS. It is critical to secure the device from unauthorized access, viruses, worms, and other malware.

**❖ Allocation Methods – Contiguous, Linked****Contiguous file Allocations method**

- Each file occupies a set of contiguous blocks on disk.
- When file is created, a disk is searched to find out a group of free memory having enough size to store a file. Once a chunk is found, required memory is allocated.
- Directory entry contains file name, starting block no. and length of a file.

|           |                       |        |
|-----------|-----------------------|--------|
| File name | Starting block number | length |
|-----------|-----------------------|--------|

- This method is used on CD-ROMs where all file sizes are known in advance. They never change during subsequent uses.



File Allocation Table

| File Name | Start Block | Length |
|-----------|-------------|--------|
| File A    | 2           | 3      |
| File B    | 9           | 5      |
| File C    | 18          | 8      |
| File D    | 30          | 2      |
| File E    | 26          | 3      |

- **Advantage**

- Simple to implement
- File access is quick

- **Disadvantage :**

- Finding free space for a new file is time consuming
- If size of an existing file is increases ,it may not be possible to accommodate such extension.
- External fragmentation may be generated

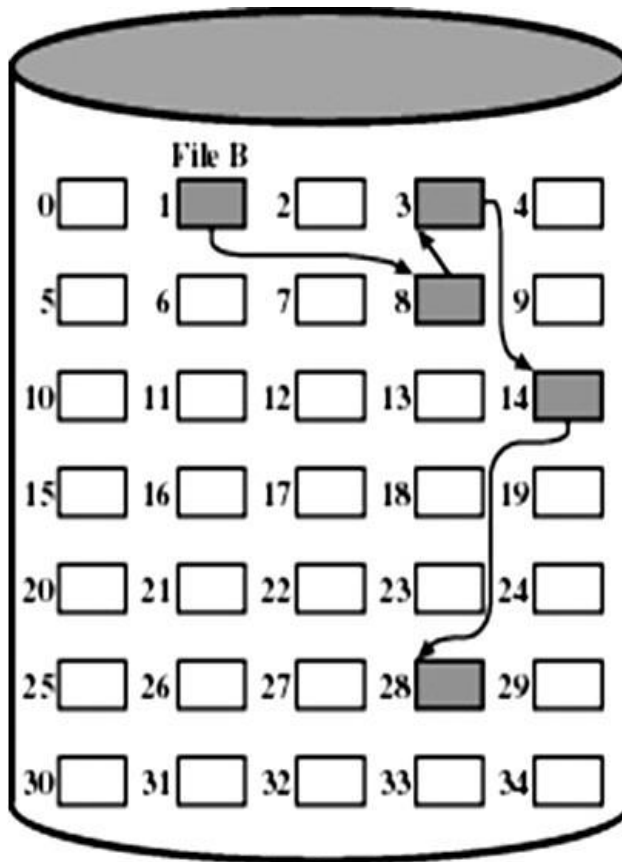
### Linked or chained allocation

- Allocation on basis of individual block
- Disk blocks are scattered any where on the disk.
- Each block contains a pointer to the next block in the chain
- Only single entry in the file allocation table
  - Starting block and length of file

|           |                       |                   |
|-----------|-----------------------|-------------------|
| File name | Starting block number | Last block number |
|-----------|-----------------------|-------------------|

- **Advantage :**

- No external fragmentation
- Disk space can be utilized effectively
- Best for sequential files



File Allocation Table

| File Name | Start Block | Length |
|-----------|-------------|--------|
| ...       | ...         | ...    |
| File B    | 1           | 5      |
| ...       | ...         | ...    |

- Linked scheme
- Multilevel index
- Combined scheme

### ❖ Disk Structure

- Hard disk is a sealed circuit. it contains one or more platters.
- **Platters :**
  - Each platter contains *magnetic coating on both of its surfaces*. Platters are stacked one on top of another.
  - Each platter contains two surfaces.
  - They rotate together round a **central spindle**.
  - **Rotation speed** is normally 3600, 5400 or 7200 rpm (Rotation per Minute).
  - Disk contains 1 to 8 platters.

- Data is read from and written to these platters using a number of **read /write heads**.

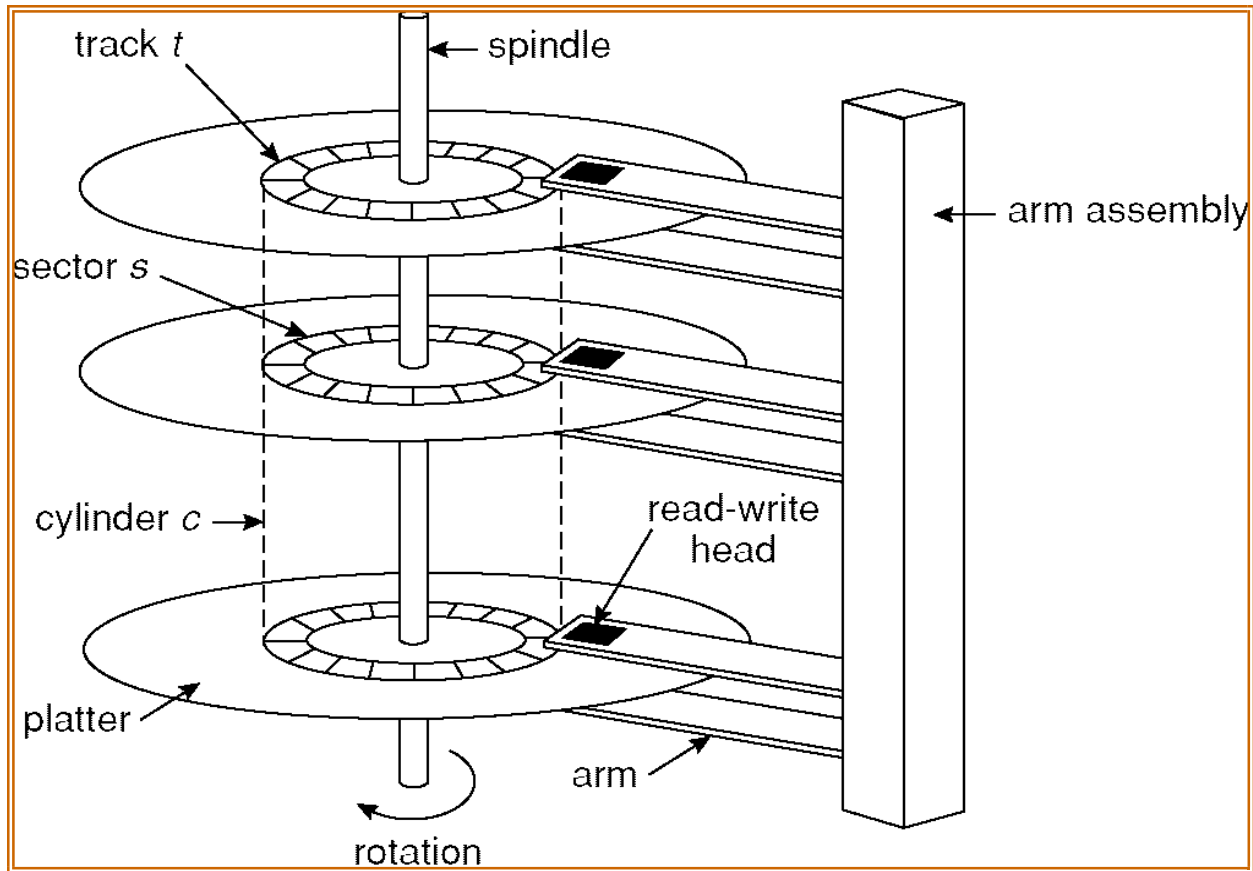
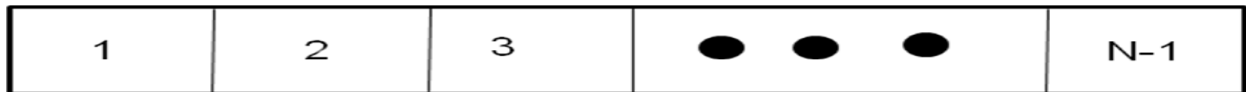


Figure :physical structure of hard disk

- There are many heads; each of head is attached to an *arm*.
- **An arm** can be moved in order to access different parts of the platters.
- A platter is a collection of circular, concentric, flat rings. These *rings* are called **tracks**.
- Data is stored on surface of a platter inside these tracks in form bits (0 to 1).there are more than thousands of tracks on a single surface.
- Each track is divided into a fixed size blocks, called *sector*. Sector size is of 512 bytes and there are hundreds of sectors on a single track.
- A group of tracks with the same radius are called **cylinders**. Each track is on different platter.

### Logical address

- Disk is considered as large one dimensional array of fixed size logical **block** .
- Size of blocks is varying from system to system.
- Unique no. is used to uniquely identify on a disk.
- Numbering starts from '0' or '1' to some maximum number.
- Total number of blocks depends upon a storage capacity of a disk and the size of block.



### ❖ Disk Scheduling

- The operating system is responsible for using hardware efficiently — for the disk drives, this means having a fast access time and disk bandwidth.
- Access time has two major components
  - **Seek time** is the time for the disk arm to move the heads to the cylinder containing the desired sector.
  - **Rotational latency** is the additional time waiting for the disk to rotate the desired sector to the disk head.
- Minimize seek time
- Seek time  $\approx$  seek distance
- Disk bandwidth is the total number of bytes transferred, divided by the total time between the first request for service and the completion of the last transfer.
- Several algorithms exist to schedule the servicing of disk I/O requests.
- We illustrate them with a request queue (0-199).

### SCAN and C-SCAN algorithm

#### Scan Algorithm

It is also called as Elevator Algorithm. In this algorithm, the disk arm moves into a particular direction till the end, satisfying all the requests coming in its path, and then it turns back and moves in the reverse direction satisfying requests coming in its path.

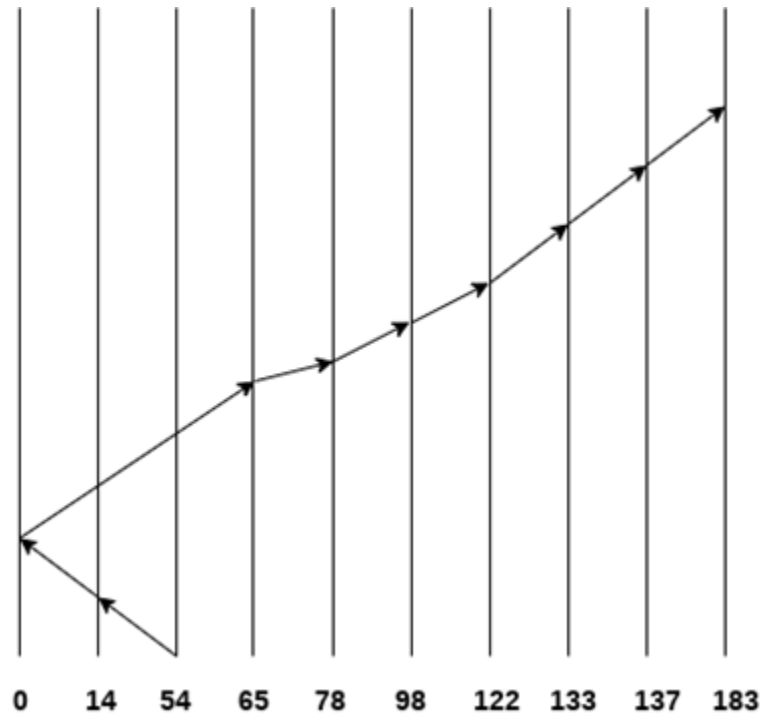
It works in the way an elevator works, elevator moves in a direction completely till the last floor of that direction and then turns back.

#### Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using SCAN scheduling.



Number of Cylinders =  $40 + 14 + 65 + 13 + 20 + 24 + 11 + 4 + 46 = 237$

### C-SCAN algorithm

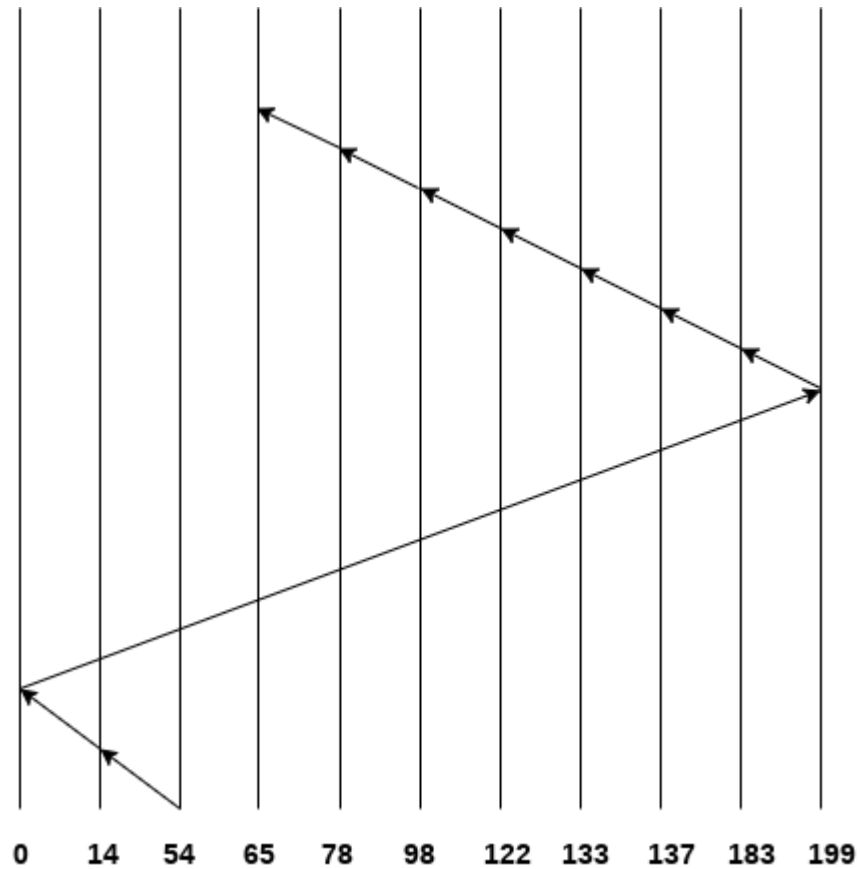
In C-SCAN algorithm, the arm of the disk moves in a particular direction servicing requests until it reaches the last cylinder, then it jumps to the last cylinder of the opposite direction without servicing any request then it turns back and start moving in that direction servicing the remaining requests.

#### Example

Consider the following disk request sequence for a disk with 100 tracks

98, 137, 122, 183, 14, 133, 65, 78

Head pointer starting at 54 and moving in left direction. Find the number of head movements in cylinders using C-SCAN scheduling.



No. of cylinders crossed =  $40 + 14 + 199 + 16 + 46 + 4 + 11 + 24 + 20 + 13 = 387$

## Unit-5 Linux Basics

### ❖ Linux Introduction

Linux was first developed by Linus torvads .

Linux is the most famous free and open source OS.

Open source Os are available in source code format rather than as compiled binary code

Free software means not by money point of view. But software that is free means it is distributed along with its source code.

Anyone who receives it is free to make change and redistribute it.

There are many versions of Linux.famous versions are,

Red hat Linux → Fedora ,Centos

Debian → Ubuntu, Linux mint, Linspire, PCLinux

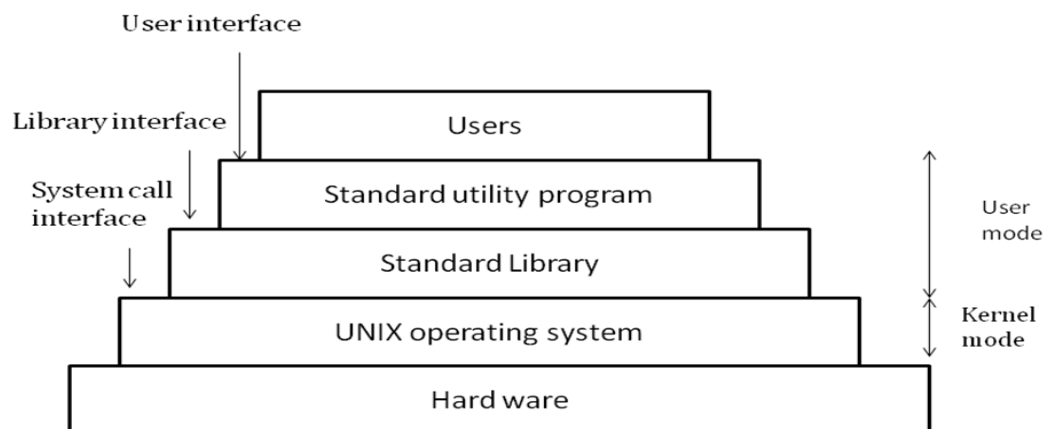
Linux can be found on wide range of devices-from personal computers,mobiles,tablets and embedded systems to main frame computers and supercomputers

### ❖ Basic architecture of Unix/ Linux

- It is also known as the layered structure of Linux.
- Linux is a UNIX-like OS, its architecture resembles to that of UNIX.

#### 1. Hardware:

- Bottom layer is hard ware.
- It consist of physical devices such as CPU ,memory ,disks ,monitors ,printers etc.
- These devices provides various services.



**Figure : Linux architecture**

**2. Linux operating system:**

- Next higher layer in Linux OS
- It is called system kernel.
- Kernel manages all the underlying hardware
- It directly interacts with the H/W and provides user programs required services
- It hides complex details of H/W
- It provides interface between user program and H/W
- Main service of OS :
  - process management , memory management ,file system management ,I/O management

**3. Standard library**

- Contains a set of procedures, one procedure per system call
- These procedures are written in assembly language and used to invoke various system calls from user program.

**4. Standard utility program**

- All UNIX versions supply OS , system call , large no. of standard program
- Program include command processor (shell) ,compilers ,editors ,text processing program, file manipulation utility a various commands...
- Program makes user tasks simpler
- User interacts with program, program interacts with OS to get services from OS

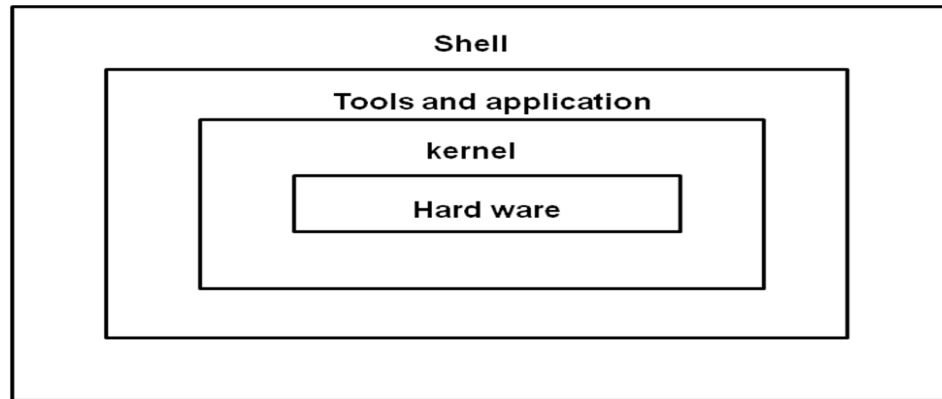
**5. users**

- Top most layers
- User program come in this layer
- They interacts with system either by library procedure to invoke system call or by using utility program such as shell

**❖ Introduction to shell and commands****Kernel**

- Kernel is core part of any OS.
- Kernel is program, which is loaded in memory ,when system is turned on and provides various services until system is turned off.
- Kernel manages all the underlying hardware
- It directly interacts with the H/W , It hides complex details of H/W
- When user needs to use any H/W ,it has to use services (system call) provided by kernel.
- System call request kernel and kernel performs the job behalf on user process.

- Main service of OS :
  - process management , memory management ,file system management ,I/O management
- Only one kernel in the system



### The Shell (command interpreter)

- A **Unix shell**, also called "the command line “
- It is interface between user programs and the kernel.
- Users direct the operation of the computer entering command input as text for a shell to execute.
- When user logs in to system processor shell starts execution
- It terminates when user logs out from system
- Shell works as command interpreter .it accepts command from user and translates them into which the kernel can understand
- More than one shell can be in system

❖ **Commands: pwd, cd, mkdir, rmdir, ls, cat, cp, rm, mv, wc, split, cmp, comm, diff, head, tail, grep, sort**

#### 1. pwd Command

The pwd command is used to display the location of the current working directory.

**Syntax:** pwd

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ pwd
/home/javatpoint
```

#### 2. mkdir Command

The mkdir command is used to create a new directory under any directory.

**Syntax:** mkdir <directory name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ mkdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

### 3. rmdir Command

The rmdir command is used to delete a directory.

**Syntax:** rmdir <directory name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ rmdir new_directory
javatpoint@javatpoint-Inspiron-3542:~$
```

### 4. ls Command

The ls command is used to display a list of content of a directory.

**Syntax:** ls

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ ls
a           Desktop          examples.desktop  Music        sample
Akash       Directory           hello.c          pico         snap
a.out       Documents           hello.i          Pictures     Templates
composer.phar Downloads           hello.o          project      Test.txt
Demo.sh     eclipse             hello.s          Public       Videos
Demo.txt   eclipse-installer   index.html       Python
Demo.txt~  eclipse-workspace  mail             Python-3.8.0
```

### 5. cd Command

The cd command is used to change the current directory.

**Syntax:** cd <directory name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ cd Desktop
javatpoint@javatpoint-Inspiron-3542:~/Desktop$
```

## 6. cat Command

The cat command is a multi-purpose utility in the Linux system. It can be used to create a file, display content of the file, copy the content of one file to another file, and more.

**Syntax:** cat [OPTION]... [FILE]..

To create a file, execute it as follows:

```
cat > <file name>
// Enter file content
```

```
cat <file name>
```

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat > Demo.txt
This is a text file.
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ cat Demo.txt
This is a text file.
```

## 7. rm Command

The rm command is used to remove a file.

**Syntax:** rm <file name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo.txt
javatpoint@javatpoint-Inspiron-3542:~/Newfolder$ rm Demo1.txt Demo2.txt
```

## 8. cp Command

The cp command is used to copy a file or directory.

**Syntax:** To copy in the same directory:

```
cp <existing file name> <new file name>
```

To copy in a different directory:

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt demo1.txt
javatpoint@javatpoint-Inspiron-3542:~$ cp demo.txt Documents
```

### 9. mv Command

The mv command is used to move a file or a directory from one location to another location.

**Syntax:** mv <file name> <directory path>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ mv demo.txt Directory
```

### 10. head Command

The head command is used to display the content of a file. It displays the first 10 lines of a file.

**Syntax:** head <file name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ head Demo.txt
1
2
3
4
5
6
7
8
9
10
```

### 11. tail Command

The tail command is similar to the head command. The difference between both commands is that it displays the last ten lines of the file content. It is useful for reading the error message.

**Syntax:** tail <file name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ tail Demo.txt
2
3
4
5
6
7
8
9
10
11
```

## 12. grep Command

The grep is the most powerful and used filter in a Linux system. The 'grep' stands for "**global regular expression print**." It is useful for searching the content from a file. Generally, it is used with the pipe.

**Syntax:** command | grep <searchWord>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ cat marks.txt | grep 9
celena-90
```

## 13. comm Command

The 'comm' command is used to compare two files or streams. By default, it displays three columns, first displays non-matching items of the first file, second indicates the non-matching item of the second file, and the third column displays the matching items of both files.

**Syntax:** comm <file1> <file2>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ comm Demo.txt Demo1.txt
      1
2
      3
comm: file 2 is not in sorted order
      4
11      5
      6
      7
      8
      9
22
33
6
7
8
9
comm: file 1 is not in sorted order
10
11
```

## 13. tr Command

The tr command is used to translate the file content like from lower case to upper case.

**Syntax:** command | tr <'old'> <'new'>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ cat marks.txt | tr 'prcu' 'PRCU'
alex-50
alen-70
jon-75
CaRRy-85
Celena-90
jUstin-80
```

#### 14. wc Command

The wc command is used to count the lines, words, and characters in a file.

**Syntax:** wc <file name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ wc marks.txt
 6  6 52 marks.txt
```

#### 15. sort Command

The sort command is used to sort files in alphabetical order.

**Syntax:** sort <file name>

**Output:**

```
javatpoint@javatpoint-Inspiron-3542:~$ sort marks.txt
alen-70
alex-50
carry-85
celena-90
jon-75
justin-80
```

❖ Editing files with “vi”, “vim”, “gedit”, “gcc”

The vi editor is elaborated as **v**isual editor. It is installed in every Unix system. In other words, it is available in all Linux distros. It is user-friendly and works same on different distros and platforms. It is a very powerful application. An improved version of vi editor is **vim**.

**The vi editor has two modes:**

- **Command Mode:** In command mode, actions are taken on the file. The vi editor starts in command mode. Here, the typed words will act as commands in vi editor. To pass a command, you need to be in command mode.
- **Insert Mode:** In insert mode, entered text will be inserted into the file. The **Esc** key will take you to the command mode from insert mode.

By default, the vi editor starts in command mode. To enter text, you have to be in insert mode, just type '**i**' and you'll be in insert mode. Although, after typing **i** nothing will appear on the screen but you'll be in insert mode. Now you can type anything.

To exit from insert mode press **Esc** key, you'll be directed to command mode.

| Commands | Action                                |
|----------|---------------------------------------|
| :wq      | Save and quit                         |
| :w       | Save                                  |
| :q       | Quit                                  |
| :w fname | Save as fname                         |
| ZZ       | Save and quit                         |
| :q!      | Quit discarding changes made          |
| :w!      | Save (and write to non-writable file) |

To exit from vi, first ensure that you are in command mode. Now, type :wq and press enter. It will save and quit vi.

### ❖ Read Command-line Arguments in Shell Scripts

Now we have developed an understanding of the command-line arguments in Linux. Now it's time to use this knowledge for practical application of the netstat command.

For this tutorial, we will go over an example to learn how to use the command-line arguments in your shell script.

First, we will create a shell script to demonstrate the working of all the reserved variables which we discussed in the previous section. Use nano or any preferred editor of your choice and copy the following.

This is the shell script which we plan to use for this purpose.

```
#!/bin/sh
echo "Script Name: $0"
echo "First Parameter of the script is $1"
echo "The second Parameter is $2"
echo "The complete list of arguments is $@"
echo "Total Number of Parameters: $# "
echo "The process ID is $$"
echo "Exit code for the script: $?"
```

Copy

Once we are done, we will save the script as PositionalParameters.sh and exit our text editor.

Now, we will open the command line on our system and run the shell script with the following arguments.

```
./PositionalParameters.sh learning command line arguments
```

Copy

The script will run with our specified arguments and use positional parameters to generate an output. If you followed the step correctly, you should see the following screen.

```
~/Desktop$ sudo sh ./PositionalParameters.sh learning command line arguments
Script Name: ./PositionalParameters.sh
First Parameter of the script is learning
The second Parameter is command
The complete list of arguments is learning command line arguments
Total Number of Parameters: 4
The process ID is 14974
Exit code for the script: 0
```

Reading Arguments

Our output shows the correct output by substituting the reserved variables with the appropriate argument when we called it.

The process was run under the process ID 14974 and quit with the exit code 0.

### ❖ The logical operators

1. **&&** :command 2 is executed only when command1 is successful in execution.

**Syntax :**

Command1 **&&** commnad2

**Example :** if [ \$a -gt \$b **&&** \$a -gt \$c ]  
echo " a is maximum"

2. **||** :command 2 is executed only when command1 is fails in its execution.command2 will not be executed at all.

**Syntax :** Command1 **||** command2

if [ ans -eq 'y' **||** ans -eq 'Y' ]  
echo " continue"

### ❖ Evaluate expression using test and [..]

- **test** and [ .....] is used to evaluate conditional expressions with integers ,strings and file attributes.

**test expression**

[ expression ]

- **Integer related test operators:**

| Expression           | Result                                                |
|----------------------|-------------------------------------------------------|
| exp1 <b>-eq</b> exp2 | It returns true if exp1 is equal to exp2              |
| exp1 <b>-ne</b> exp2 | It returns true if exp1 is not equal to exp2          |
| exp1 <b>-le</b> exp2 | It returns true if exp1 is less than or equal to exp2 |
| exp1 <b>-lt</b> exp2 | It returns true if exp1 is less than to exp2          |
| exp1 <b>-ge</b> exp2 | It returns true if exp1 is greater or equal to exp2   |
| exp1 <b>-gt</b> exp2 | It returns true if exp1 is greater than exp2          |

- **File related test operators**

| Expression     | Result                                              |
|----------------|-----------------------------------------------------|
| <b>-b file</b> | True if file exists and is a block special file     |
| <b>-c file</b> | True if file exists and is a character special file |

|                  |                                                       |
|------------------|-------------------------------------------------------|
| <b>-d file</b>   | True if file exists and is a directory                |
| <b>-f file</b>   | True if file exists and is a regular file             |
| <b>-r file</b>   | True if file exists and is a readable                 |
| <b>-w file</b>   | True if file exists and is a writable                 |
| <b>-x file</b>   | True if file exists and is a executable               |
| <b>-k file</b>   | True if file exists and is a sticky bit is set        |
| <b>-l file</b>   | True if file exists and is a symbolic link            |
| <b>-s file</b>   | True if file exists and its size is greater than zero |
| <b>f1 -nt f2</b> | True if f1 is newer than f2                           |
| <b>f1 -ot f2</b> | True if f1 is older than f2                           |
| <b>f1 -ef f2</b> | True if f1 is linked to f2                            |

- String related test expression

| Expression                | Result                                |
|---------------------------|---------------------------------------|
| <b>-n</b>                 | True if length of string is not zero  |
| <b>-z</b>                 | True if length of string is zero      |
| <b>String</b>             | True if string is not set to null     |
| <b>string1 = string2</b>  | True if string1 is equal to string2   |
| <b>string1 != string2</b> | True if sting1 is not equal to sting2 |
| <b>string = pattern</b>   | True if string matches pattern        |
| <b>String != pattern</b>  | True if string does not match pattern |

### ❖ Branching : if, case

#### 1. if

Syntax:                    **if** command1

**then**

**commands**

**fi**

**2. if ....else**

Syntax :

```
if command1
then
    command2
else
    command3
fi
```

**3. else ...if ladder :**

Syntax :

```
if command-1
then
    commands
elif command-2
then
    commands
elif command-3
then
    commands
else
    commands
fi
```

**4. case**

```
case expression in
pattern1)  commands1      ;;
pattern2)  commands2      ;;
pattern3)  commands3      ;;
.....
.....
esac
```

**❖ Basic of Looping : while, for****1. while**

Syntax :

```
while command1
do
    commands
done
```

**2. until :**

Syntax :

```
until command1
```

**do**

commands

**done**

**3. for :**

**Syntax :**

**for** i **in** list

**do**

commands

**done**